课 程 内 容

第 1 章 编 译 引 论

第 2 章 形式语言与自动机基础

第 3 章 词法分析 (Lexical Analysis)

第 4 章 语法分析(Syntax Analysis)
—— 自上而下分析法

第 5 章 语法分析(Syntax Analysis)
—— 自下而上分析法

第 6 章 语义分析与中间代码生成

第 7 章 运 行 环 境

第 8 章 代码优化（optimization）

第 6 章　语义分析与中间代码生成

6.1　语义分析综述

6.2　语法制导翻译与属性文法

6.3　中间语言

6.4　符号表

~~6.5　类型检查（自学）~~

6.6　语句翻译与中间代码生成

语义分析的地位

Front end (分析)	语义处理	Back end (综合)

- 编译程序最实质性的工作；
- 第一次对源程序的语义作出解释，引起源程序质的变化。

- 语义分析的任务

按照语法分析器识别的语法范畴进行语义检查和处理，产生相应的中间代码或目标代码。

- 中间代码

介于源语言和目标代码之间的一种代码。

- 引入中间代码的目的

1. 方便生成目标代码；
2. 便于优化；
3. 便于移植。

第 6 章　语义分析与中间代码生成

6.1　语义分析综述

6.2　语法制导翻译与属性文法

6.3　中间语言

6.4　符号表

~~6.5　类型检查（自学）~~

6.6　语句翻译与中间代码生成

- **语法制导翻译**

为文法的每一个产生式配一个相应的语义子程序（或语义规则描述的语义动作），并在语法分析的同时调用它。

- **属性翻译文法**

把语义引入文法，给产生式中的文法符号附加“属性”，构成涉及语义的翻译文法。

文法符号的语义性质(语义属性)

- **形式定义** $A = (G, V, F)$

其中：

G：一般为二型文法；

V：属性的有穷集；

F：关于属性的断言或谓词的有穷集。

- 属性表示 数、符号串、类型、存储空间…

$N.t$ （N是文法符号，t是N的属性）

（终结符的属性均为lexval）

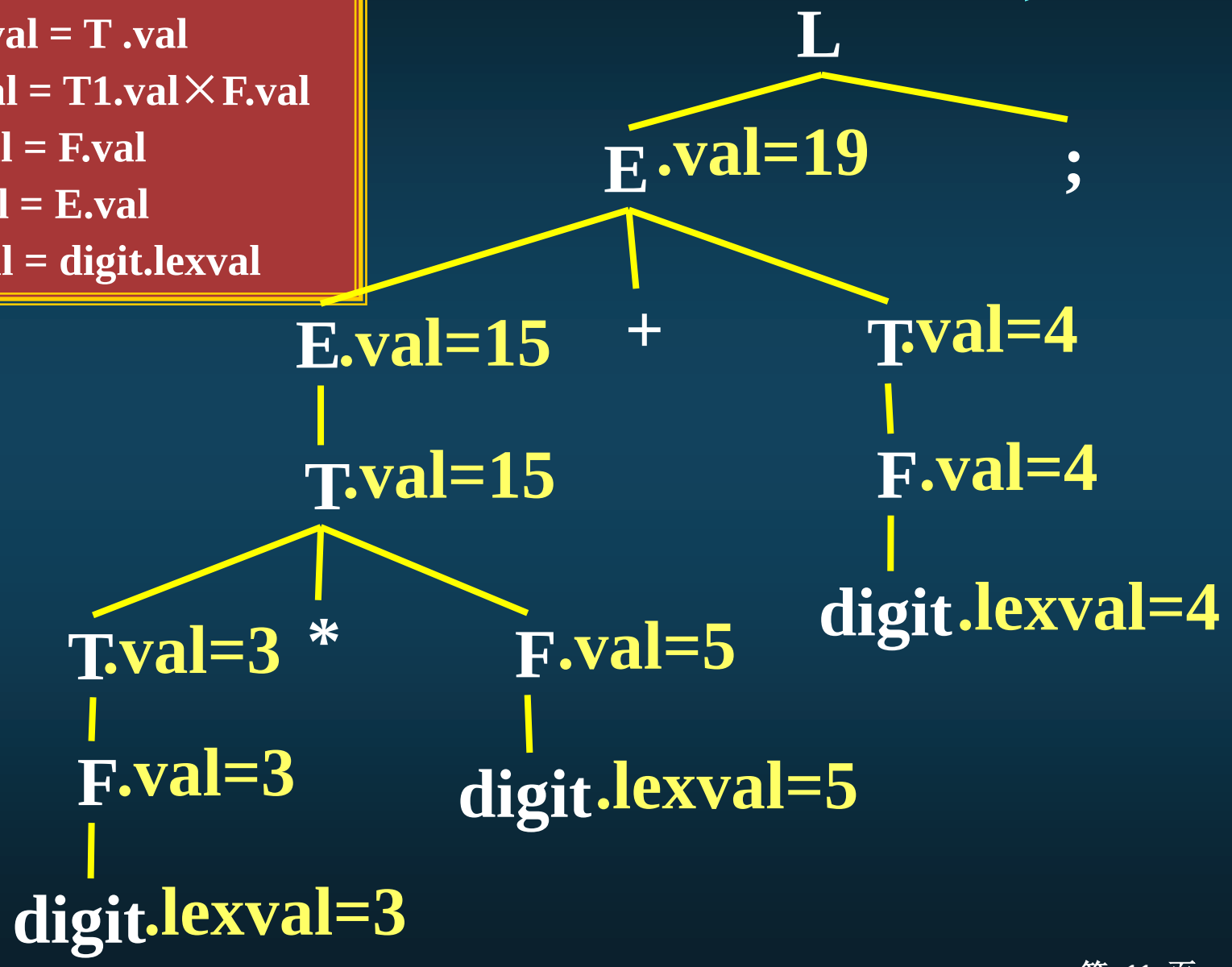
产生式	语义规则
$L \rightarrow E$	print(E .val)
$E \rightarrow E + T$	E .val = E_1 .val + T .val
$E \rightarrow T$	E .val = T .val
$T \rightarrow T * F$	T .val = T_1 .val $\times$ F .val
$T \rightarrow F$	T .val = F .val
$F \rightarrow (E)$	F .val = E .val
$F \rightarrow$ digit	F .val = digit .lexval

- 综合属性

分析树中，如果一个结点的属性值是通过子结点的属性值计算得到，则称为综合属性。

$L \to E;$	print(E .val)
$E \to E+T$	E.val = E1.val + T.val
$E \to T$	E .val = T .val
$T \to T*F$	T.val = T1.val×F.val
$T \to F$	T.val = F.val
$F \to (E)$	F.val = E.val
$F \to$digit	F.val = digit.lexval

例如， 3 * 5 + 4 ;



- **综合属性**

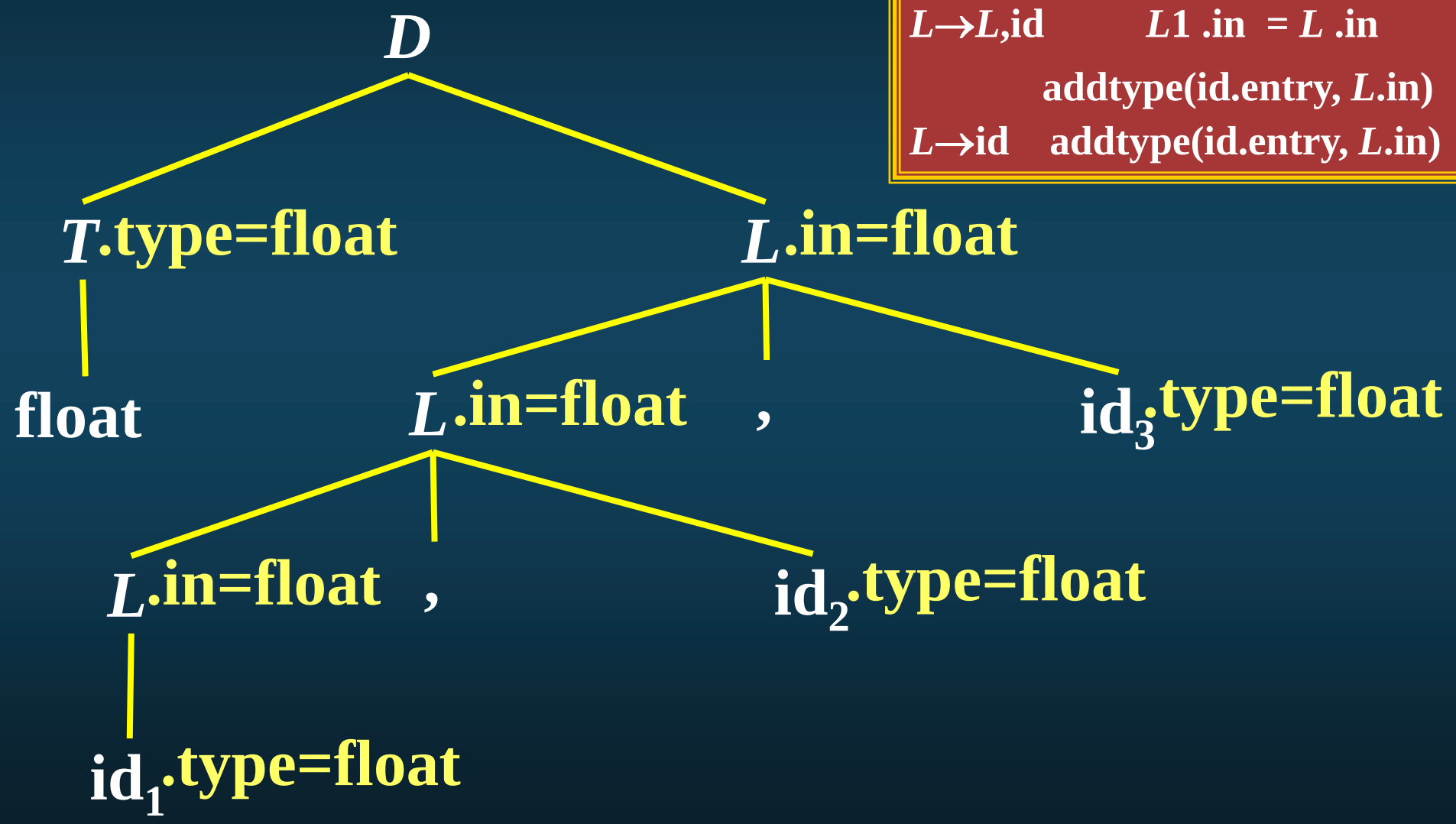
分析树中，如果一个结点的属性值是通过子结点的属性值计算得到则称为综合属性。

- **继承属性**

分析树中，如果一个结点的属性值是由该结点的父结点和(或)兄弟结点的属性定义的称为继承属性。

例如，float id_1，id_2，id_3

$D \rightarrow TL$	L.in = T.type
$T \rightarrow$ int	T.type = integer
$T \rightarrow$ float	T.type = float
$L \rightarrow L$,id	$L1$.in = L.in addtype(id.entry, L.in)
$L \rightarrow$ id	addtype(id.entry, L.in)



第 6 章　语义分析与中间代码生成

6.1　语义分析综述

6.2　语法制导翻译与属性文法

6.3　中间语言

6.4　符号表

~~6.5　类型检查（自学）~~

6.6　语句翻译与中间代码生成

中间语言(中间代码)是语义程序的输出。

中间语言的设计与应用既要考虑从源语言到目标语言的翻译跨度又要考虑目标机的指令集特点。

中间语言
- N元式(三元式、间接三元式四元式)
- 逆波兰式
- 图

一. 逆波兰式 (后缀式)

形式定义: $\underbrace{e_1 e_2 \ldots e_n}_{\text{运算对象}} \theta$ $(n \geq 1)$

θ → 运算符

例如,

$a * b \Rightarrow ab*$

$- a \Rightarrow a@$ (@表示单目一)

$a+b*(c+d)*(e+f) \Rightarrow a\ b\ c\ d\ +\ *\ e\ f\ +\ *\ +$

Label1

IF<expr>then<s1>else<s2>

Label2

编号

(1)<expr> <4> J_F

(2)<s1>

(3)<5> J

(4)<s2>

(5)

引入两个操作符：J_F、J。

J_F：二目操作符，如果第一运算对象的值为假，则产生一个到第二运算对象的转移。

J：单目操作符，产生一个到运算对象的转移。

例6.1 设有如下C程序片段

```
{
    int  k;
    i=j=0;
h:  k=100;
    if (k>i+j)
        { k--;  i++; }
    else
        k=i*2-j*2;
    goto h;
}
```

逆波兰式代码：

(1) i j 0 = =

(2) k 100 =

(3) k i j + > (7) J_F

(4) k k 1 – =

(5) i i 1 + =

(6) (8) J

(7) k i 2*j2*– =

(8) (2) J

编号

二. N元式 N个域的记录结构

$$(\underbrace{D_1}_{\text{OP域}},\ \underbrace{D_2,\ D_3,\ \ldots,\ D_n}_{\text{操作对象域}})$$

- 常见N元式

三元式、间接三元式 ⟷ 双地址机指令形式

四元式 ⟷ 三地址机指令形式

三元式形式定义：

$$NO.\ (OP,\ ARG_1,\ ARG_2)$$

其中：

NO. :为产生的三元式的顺序编号；

OP : 是操作符；

ARG1和ARG2为第一操作数和第二操作数。（也可以是前面某一个三元式的编号， 代表该三元式的计算结果被作为操作数)

间接三元式形式定义：

间接码表 + 三元式表

例如，语句 X=A+B*C 的三元式表示为

NO.	OP	ARG1	ARG2
(1)	*	B	C
(2)	+	A	(1)
(3)	=	X	(2)

if X>Y then Z = X else Z = Y+1

的三元式可以表示为

NO.	OP	ARG1	ARG2
(1)	>	X	Y
(2)	J_F	(1)	(5)
(3)	=	Z	X
(4)	J		(7)
(5)	+	Y	1
(6)	=	Z	(5)
(7)			

$(a+b)^2 + (a+b)^3$

NO	OP	ARG1	ARG2
①	+	a	b
②	↑	①	2
③	+	a	b
④	↑	③	3
⑤	+	②	④

其中：“↑”表示幂运算。

$(a+b)^2 + (a+b)^3$ 的间接三元式

控制三元式代码执行顺序

间接码表

①
②
①
③
④

三元式表

①	+, a, b
②	↑, ①, 2
③	↑, ①, 3
④	+, ②, ③

四元式形式定义：

NO. (OP， ARG_1 ， ARG_2 ， Result)

$(a+b)^2 + (a+b)^3$的四元式：

(0)	+ ,	a,	b,	T_1
(1)	↑ ,	T_1,	2,	T_2
(2)	+ ,	a,	b,	T_3
(3)	↑ ,	T_3,	3,	T_4
(4)	+,	T_2,	T_4 ,	T_5

** 注：T_i是临时变量。

if X>Y then Z = X else Z = Y+1

的四元式可以表示为

NO.	OP	ARG1	ARG2	Result
(1)	>	X	Y	t1
(2)	J_F	t1		(5)
(3)	=	X		Z
(4)	J			(7)
(5)	+	Y	1	t2
(6)	=	t2		Z
(7)				

三. 图（树）

一个三元式 ⟷ 一棵子树

OP ⟷ 子树根

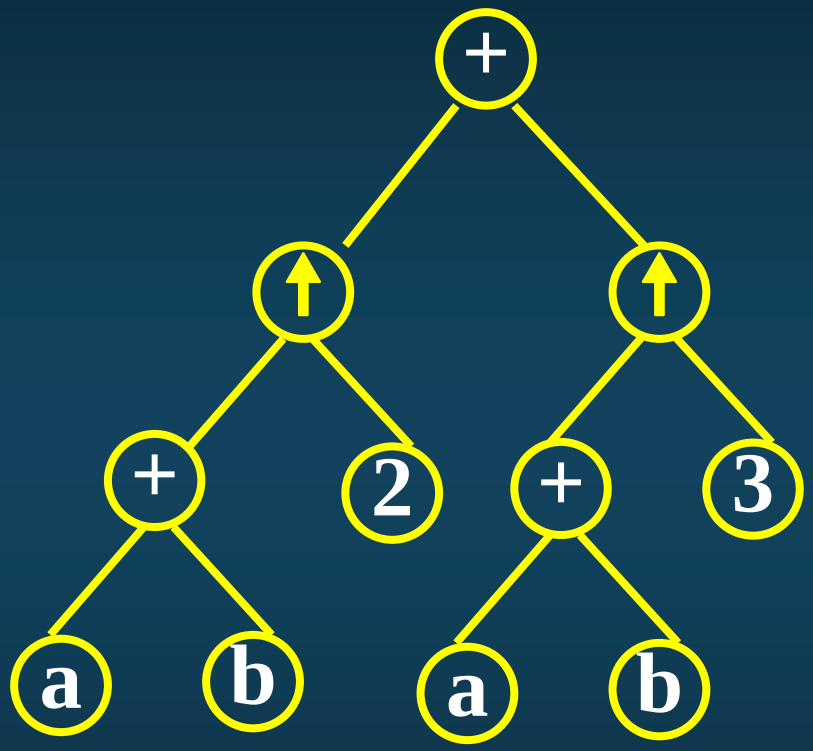
ARG1 ⟷ 子树左叶节点

ARG2 ⟷ 子树右叶节点

$(a+b)^2 + (a+b)^3$

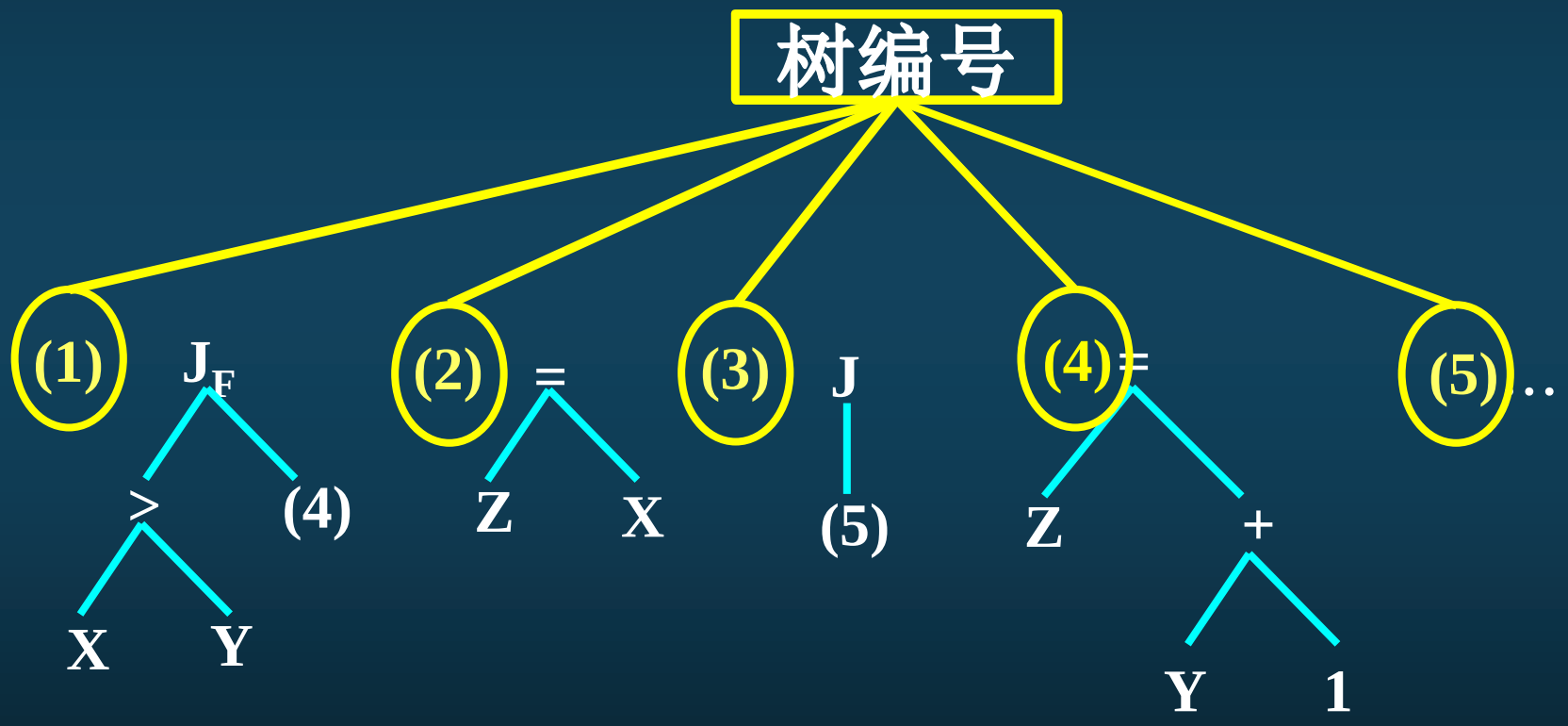
三元式

①	+ , a, b
②	↑ , ① , 2
③	+ , a, b
④	↑ , ③ , 3
⑤	+, ② , ④

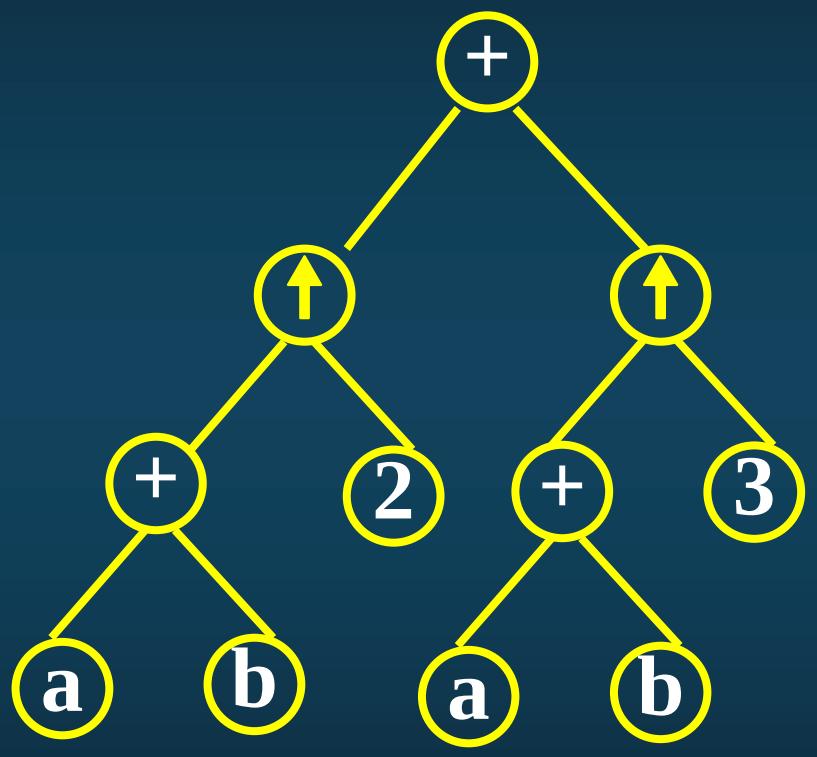


if X>Y then Z = X else Z = Y+1

的树可以表示为



几种表示间关系



树

↓ 线性化：后序遍历

逆波兰式

↓ “单步”表示

三元式

综述：

语法分析的结果为树；

逆波兰式适于栈式存储的计算机（堆栈机）；

四元式便于优化；

间接三元式优化时的时空效率类似于四元式。

第 6 章　语义分析与中间代码生成

6.1　语义分析综述

6.2　语法制导翻译与属性文法

6.3　中间语言

6.4　符号表

~~6.5　类型检查（自学）~~

6.6　语句翻译与中间代码生成

- 源程序流基本组成单位特点

关键字：表示语句性质，反映语句结构；

标识符：表示程序中各种实体；

如，变量名；常量名；标号名；过程名；函数名；文件名，数组名 …

V: name , type , addr , level …

array: name , type , addr , dim , level , size …

语句标号：name , addr , 定义否 …

过程、函数：name , addr , 形参 …

反映了标识符的语义特征属性，是翻译的依据。

记录于符号表中（namelist）

整个编译过程中动态地采集、记录、变更、引用

例如，设有C程序片段

```
        :
  int  i , a[4] ;
{  …
   i=a[2] ;
   …
}
```

词法语法语义分析

i, a ⇒ 符号表

i.type ⇒符号表

a.type ⇒符号表

a.维数 ⇒ 符号表

a.每维大小 ⇒ 符号表

……

类型检查;

数组越界检查;

回填addr; …

- 符号表

存放源程序中有关标识符的属性信息的数据结构。

- 符号表结构

名字域	属性信息域

- 符号表作用
  - 收集标识符属性信息；
  - 语义检查依据；
  - 代码生成时地址分配依据。

- 常见标识符类的主要属性与作用

1. 标识符名。作用：查重(考虑作用域和可视性前提)
2. 类型。作用：存储空间分配；可施加运算的检查等
3. 存储类别。作用：提供语义处理、检查、存储分配的依据。
4. 作用域、可视性。作用：动态活动环境支持，提供量所在层次；

第 6 章　语义分析与中间代码生成

6.1　语义分析综述

6.2　语法制导翻译与属性文法

6.3　中间语言

6.4　符号表

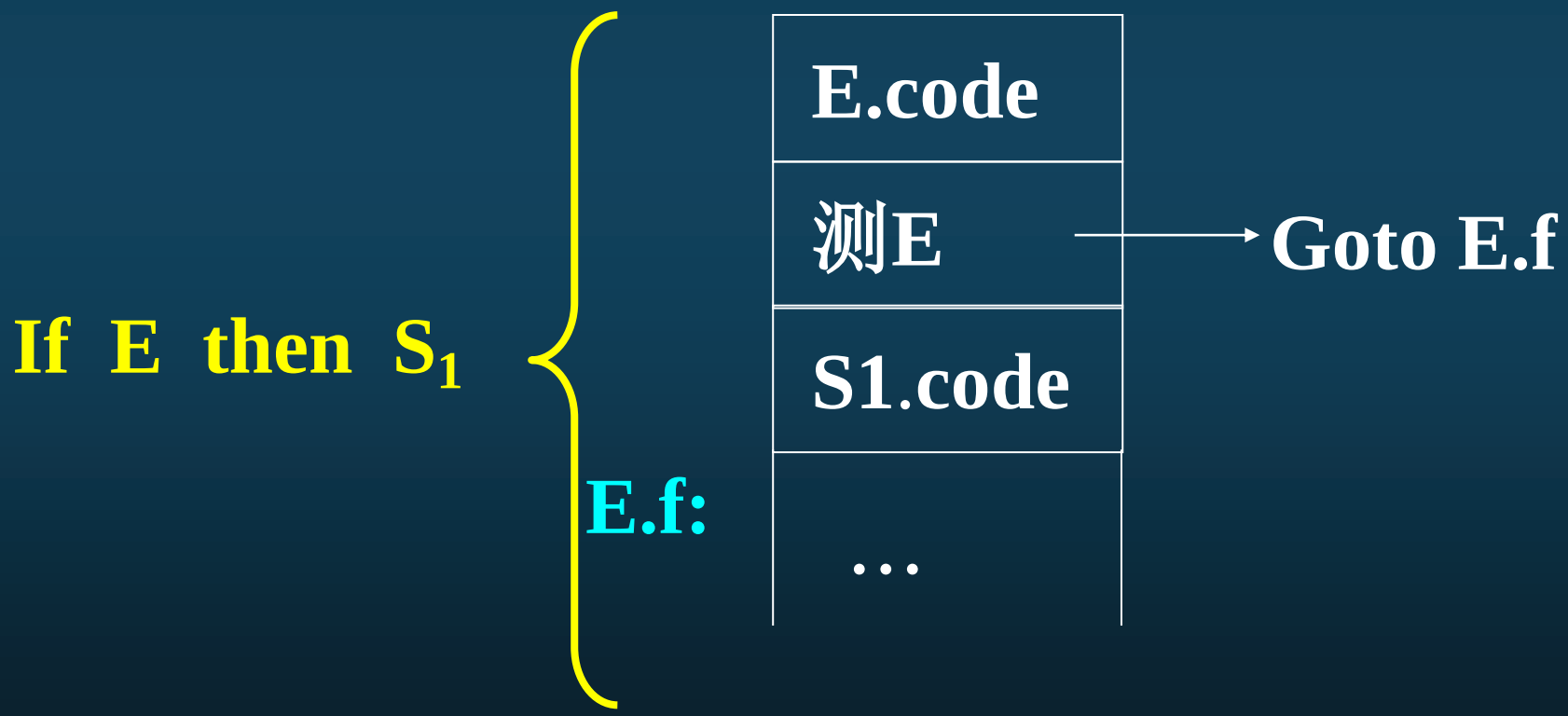
~~6.5　类型检查（自学）~~

6.6　语句翻译与中间代码生成

- 语句翻译设计要点

1. 根据语义确定语句的目标结构；

源语句 ⇒ 中间及目标代码的布局



- 语句翻译设计要点

1. 根据语义确定语句的目标结构；
2. 确定中间代码；
3. 根据目标结构和语义规则，构造语义子程序（转换翻译程序）；
4. 涉及的实现技术

6.6 语句翻译与中间代码生成

6.6.1 说明类语句的翻译

6.6.2 赋值语句与表达式翻译

6.6.3 控制流类语句翻译

6.6.4 数组说明与数组元素引用的翻译

6.6.5 过程、函数说明和调用的翻译

- 说明类语句

语言中定义性信息，一般不产生目标代码，其作用是辅助完成编译。

例如， 常量说明，变量说明，类型说明，对象说明，标号说明 ……

- 说明类语句的处理

相关说明的属性信息填入符号表，提供语义检查和存储分配的依据。

6.6.1 说明类语句的翻译

一. 常量定义语句的翻译

二. 简单说明类语句

三. 复合类型说明语句

一. 常量定义语句的翻译

常量说明语句的文法

CONST_DEF → CONST <con_list>;

<con_list> → <con_list>;<CD>

<con_list> → <CD>

<CD> → id=num

常量说明语句的语义子程序

```
{ num.ord=look_con_table(num.lexval);
    id.ord=num.ord;
   id.type= num.type;
   id.kind=constant;
 add(id.entry;id.ord; id.type; id.kind) }
```

举例说明：

CONST 标识符＝常量；…

CONST pi=3.1416; true=1;

CONST pi=3.1415926;

namelist

name	kind	type	addr	
pi	CONS	R		
true	CONS	B		
…				

conslist

value
3.1416
1
3.1415926

6.6.1 说明类语句的翻译

一. 常量定义语句的翻译

二. 简单说明类语句

三. 复合类型说明语句

二. 简单说明类语句

- 翻译方案和语义子程序

D→int id { fill(ENTRY(id)，int)；D.AT=int }

D→float id { fill(ENTRY(id)，float)；D.AT=float }

D→D，id; { fill(ENTRY(id)，D1 .AT)；
D.AT=D1 .AT }

* 其中：

D.AT： 非终结符D的属性。

fill(P, A): 完成把性质A填入P所指的符号表入口的相应数据项中的函数。

ENTRY(i)： 给出i所代表的量在符号表中的入口的函数

举例说明：

int i , j ;

float a ,b ;

类型

变量表

namelist

name	kind	type	addr	
i	V	I	0	
j	V	I	4	…
a	V	R	8	
b	V	R	16	
…				

6.6.1 说明类语句的翻译

一. 常量定义语句的翻译

二. 简单说明类语句

三. 复合类型说明语句

三. 复合类型说明语句

$$T \rightarrow struct\ \ L\{D\}\ V$$
$$L \rightarrow id \mid \varepsilon$$
$$D \rightarrow D;F \mid F$$
$$F \rightarrow type\ V;$$
$$V \rightarrow V, id \mid \varepsilon$$

其中：

L: 结构类型名； **D:** 结构成员； **V:** 变量表；

F: 结构成员项； **id:** 标识符；

例如，

```
struct date
    { int year, month, day;}  today, yesterday;
```

- 语义处理涉及

  (1) 结构成员与该结构相关；

  (2) 结构的存储：一个结构的所有成员项连续存放(简单方式)；

  (3) 结构的引用是结构成员的引用，不能整体引用；（成员信息须单独记录）

namelist

name	kind	…	addr
k1	struct		
…			

例如，

```
struct k1{
    int  a;
    float  b;
    int c[10];
    char  d;
}
```

结构成员分表

name	kind	type	LEN	OFFSET
a	V	I	4	0
b	V	R	8	4
c	array	I	40	12
d	V	C	1	52
…				

第 6 章 语义分析与中间代码生成

6.6.1 说明类语句的翻译

6.6.2 赋值语句与表达式翻译

6.6.3 控制流类语句翻译

6.6.4 数组说明与数组元素引用的翻译

6.6.5 过程、函数说明和调用的翻译

- 赋值语句形式定义

$$A \rightarrow V = E$$

- 赋值语句目标结构

** 赋值语句的处理集中在表达式的处理上

▪ 赋值语句翻译语义子程序

A →i=E

{ GEN(=，E.PLACE， _，ENTRY(i) }

其中:

GEN(OP，ARG1，ARG2，RESULT): 函数。

把四元式(OP，ARG1，ARG2，RESULT)填入四元式表。

E. PLACE：表示存放E值的变量名在符号表的入口地址。

ENTRY(i)：给出i表示的单词在符号表中的入口。

- 赋值语句语义处理

(1) 表达式处理(产生表达式的中间代码)；

(2) “=”的处理： “=”左右部类型相容性检查和转换；

- 表达式形式定义

$$E \rightarrow E\ op\ E \mid op\ E \mid id$$

其中：

OP：为运算符；

E, id：运算对象。

表达式翻译的语义子程序

(1) E →E OP E

{ E. PLACE=NEWTEMP;

GEN（OP, E_1. PLACE, E_2. PLACE, E.PLACE）}

(2) E →OP E

{ E. PLACE=NEWTEMP;

GEN(OP, E1. PLACE， _， E.PLACE）}

(3) E →id

{ E. PLACE=ENTRY(id) }

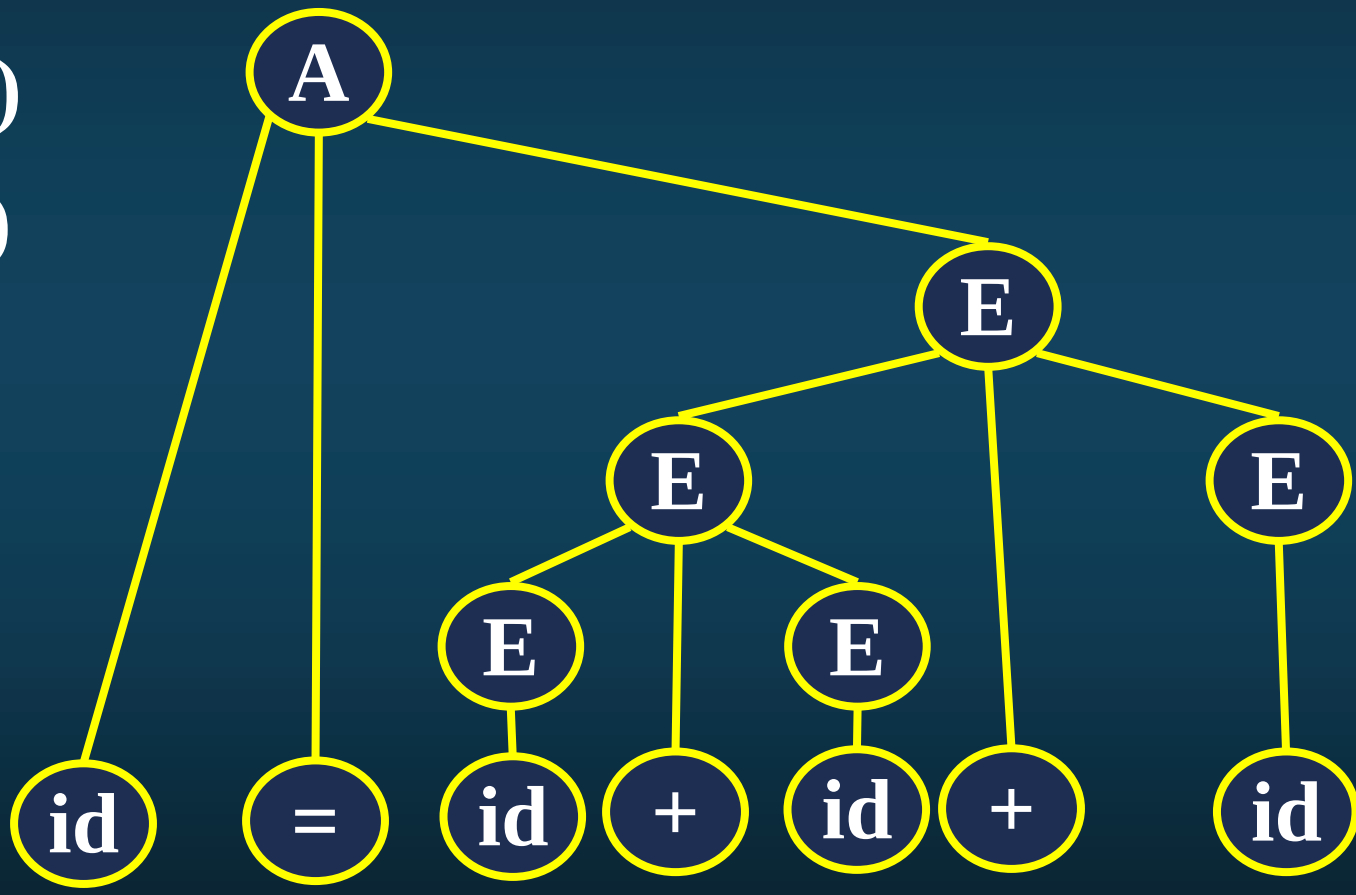
赋值 语句与表达式翻译举例：

写出语句a=b+c+d翻译后的四元式代码：

(+ b c t1)

(+ t1 d t2)

(= t2 -- a)



第 6 章 语义分析与中间代码生成

6.6.1 说明类语句的翻译

6.6.2 赋值语句与表达式翻译

6.6.3 控制流类语句翻译

6.6.4 数组说明与数组元素引用的翻译

6.6.5 过程、函数说明和调用的翻译

- **控制流语句:** 改变程序执行顺序，引起程序执行发生跳转的语句（向前或向后）。
  - 程序设计语言中出现频繁的语句；
  - 为可执行语句，要产生相应的目标代码；
  - 控制流程的变换，依靠代码中的跳转指令与对应跳转的语句标号。

- 跳转目标与依据

语句标号
- 显式：位于源语句之前；（如，L1： S;）
- 隐式：含于源语句之中但未标识

If (e_r) S_1; else S_2

跳出if 继续

e_r=T 跳转目标

e_r=F 跳转目标

循环变量重赋值

for (e_1; e_2; e_3) S ;

循环体终止判别

循环体开 始

跳出循环体继续

6.6.3 控制流类语句翻译

一. 语句标号与拉链返填技术

二. 条件语句的翻译

三. 循环语句的翻译

四. 多分支语句的翻译

- 语句标号处理

控制流类语句处理面对的公共问题和实现技术

- 语句标号处理
  - 先定义后引用
  - 先引用后定义

语句标号

- 定义性出现 （如， L1: S ;）
- 使用性出现 （如， goto L1 ;）

语句标号表（LT）

标号名	定义否(flag)	addr

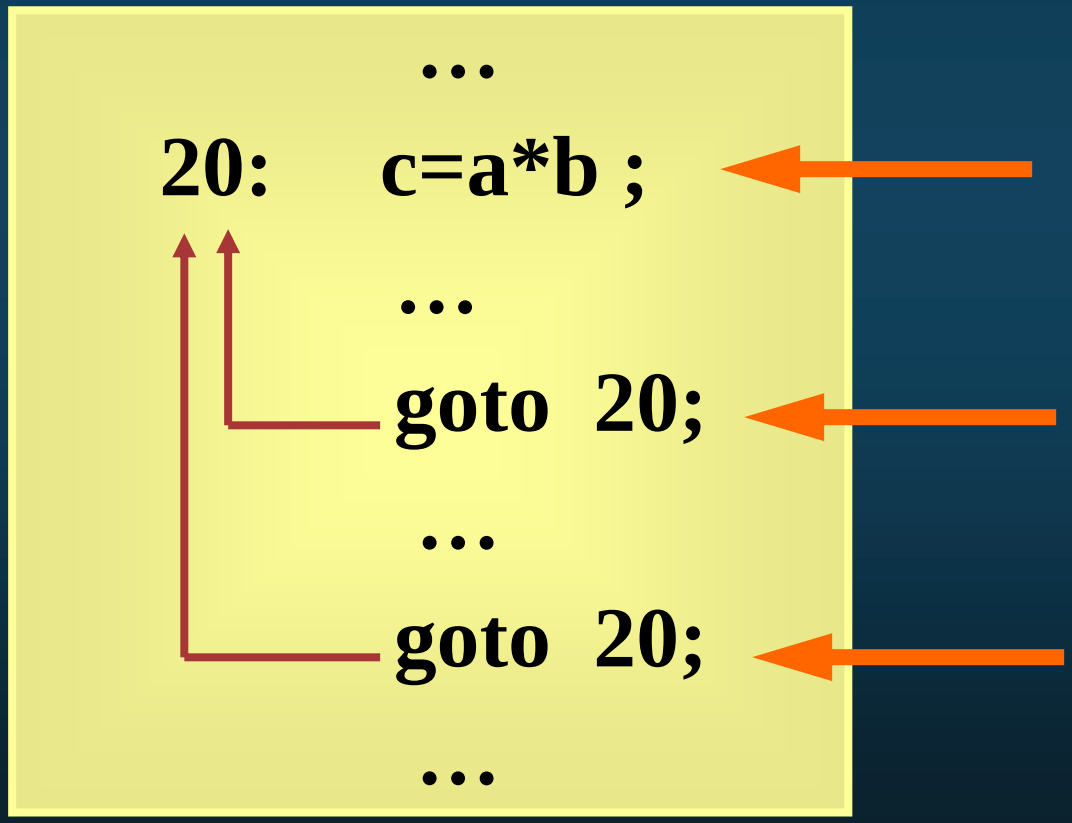
1表示定义性出现
0表示使用性出现

标号所标识的源语句的中间代码序列的第一条代码的存放地址

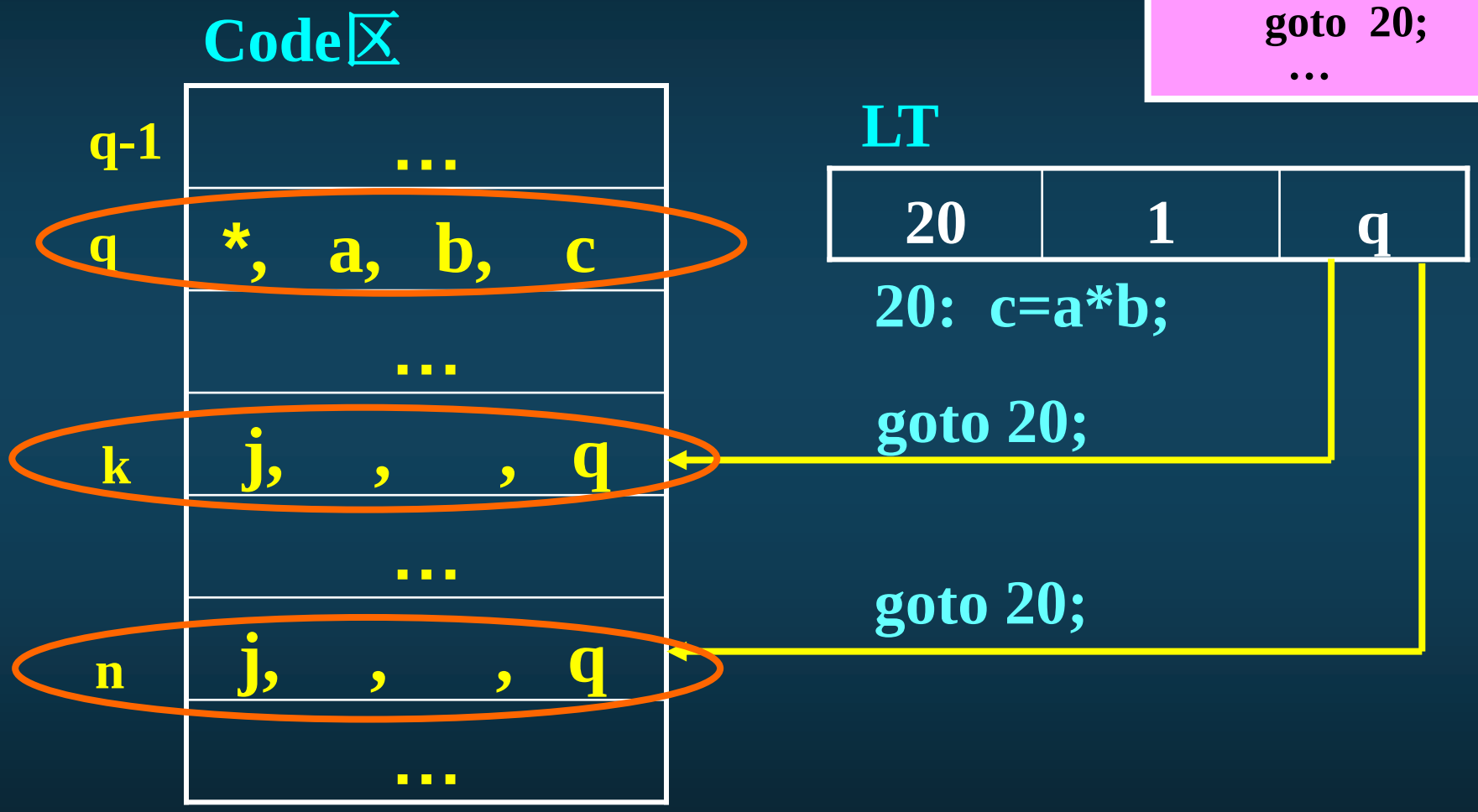
- 对多遍扫描的编译器

  视为一种情况(先定义后引用) 处理。

例如，



```
20:   c=a*b ;
      …
      goto  20;
      …
      goto  20;
      …
```



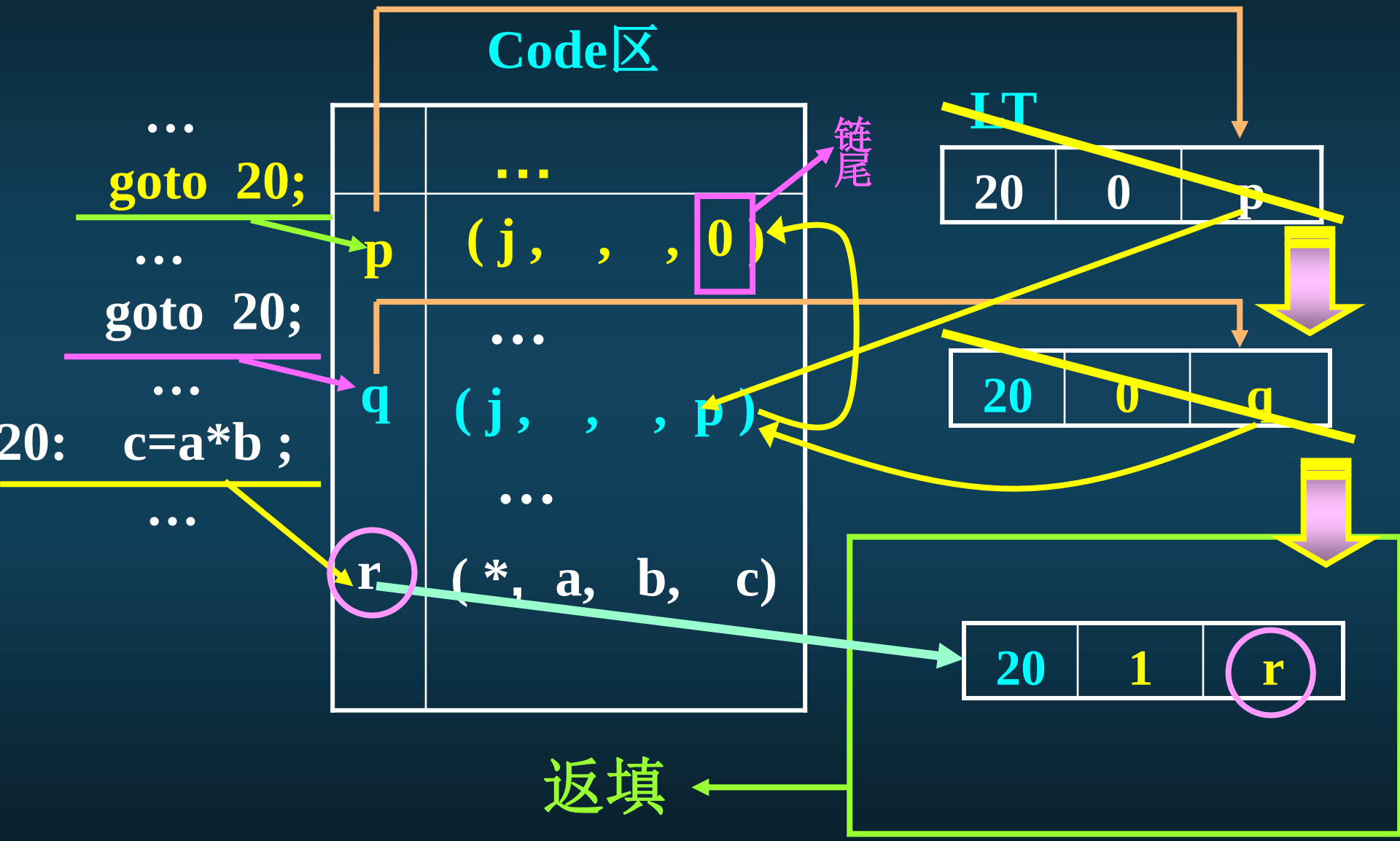
- 拉链-返填技术处理标号先引用后定义的情况：

适用于一遍扫描的编译器；

例如，

```
…
goto  20;
…
goto  20;
…
20:   c=a*b ;
…
```

产生这2条语句的代码时，还未扫描到20号语句，即没有生成该语句的代码，因此转向的目标地址未定。


Code区
…
goto 20;
…
goto 20;
…
20: c=a*b ;
…
p
q
r
…
(j , , , 0)
…
(j , , , p)
…
(*, a, b, c)
链尾
LT
20 0 p
20 0 q
20 1 r
返填

…
goto 20;
…
goto 20;
…
20: c=a*b ;
…

Code区

	…
p	(j , , , r)
	…
q	(j , , , r)
	…
r	(*, a, b, c)

LT

20	0	p
20	0	q
20	1	r

拉链

返填

✌ **注意：**

1. 链尾标志在代码区的四元式中，链头在标号表中，链在与同一语句标号相关的跳转代码中；

2. 拉链次序：

p ⟶ q

尾 头

3. 返填次序：

q ⟶ p （在代码区跳转指令中(addr=0) ）

头 尾

一. 语句标号与拉链返填技术

二. 条件语句的翻译

三. 循环语句的翻译

四. 多分支语句的翻译

二. 条件语句的翻译

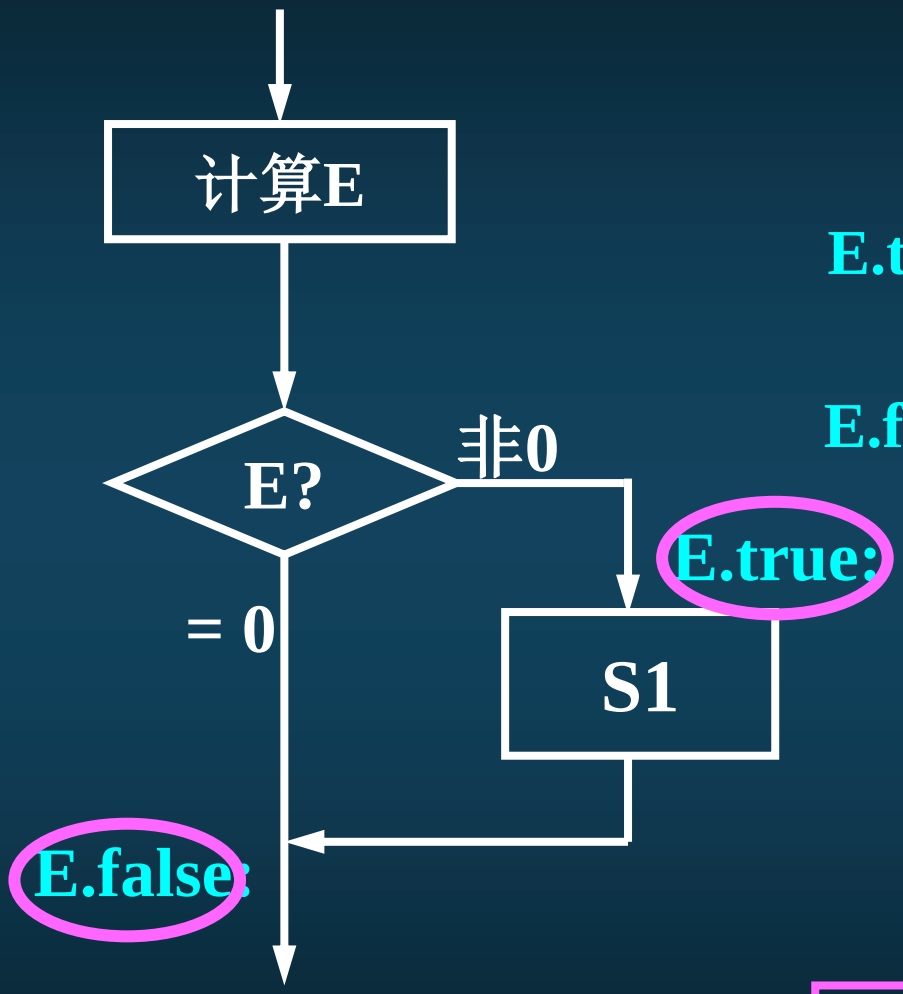
条件语句文法:

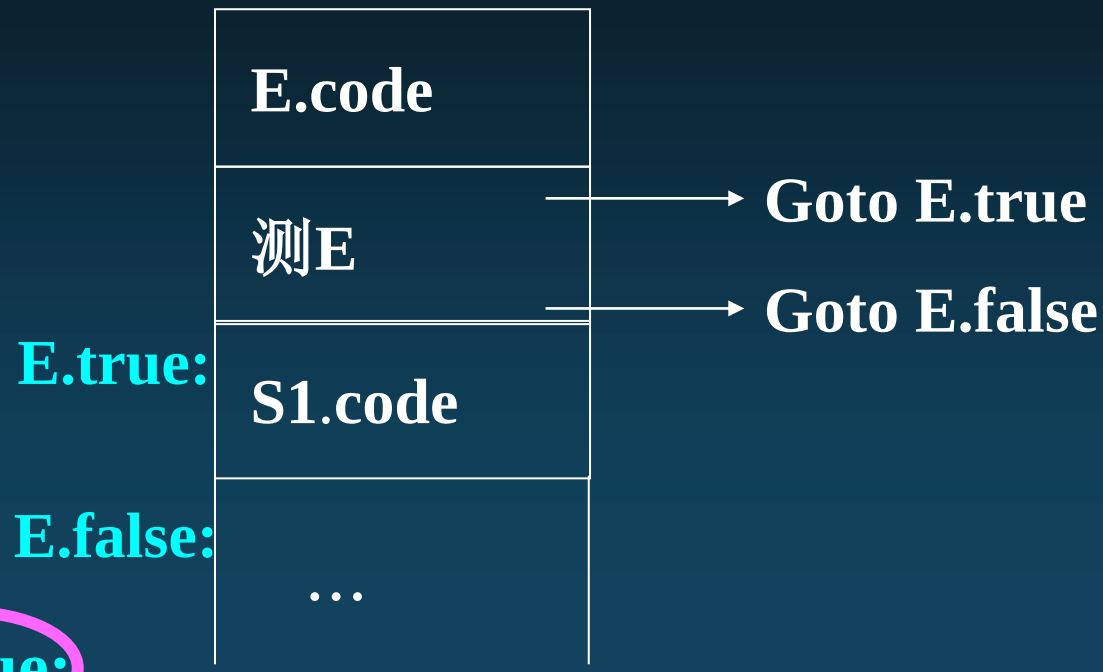
S → if (E) then S1 ① |
if (E) S1 else S2 ② |
while (E) S1 ③

其中:

E: 条件表达式 (有语义值 E.True和 E.False);

S1, S2: 语句;





E.true E.false

S → if (E) then S1

- 条件语句①翻译的语义子程序

S →if E then S1

{ E.true=newlabel;

E.false=newlabel;

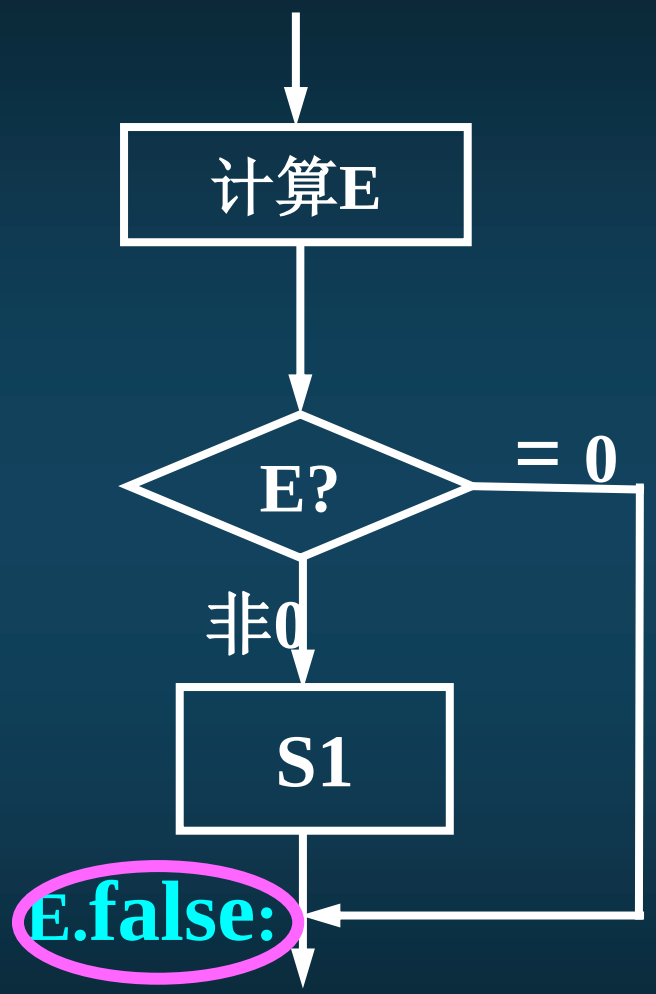
S.code=E.code|GEN(jT， _， _， E.true)|

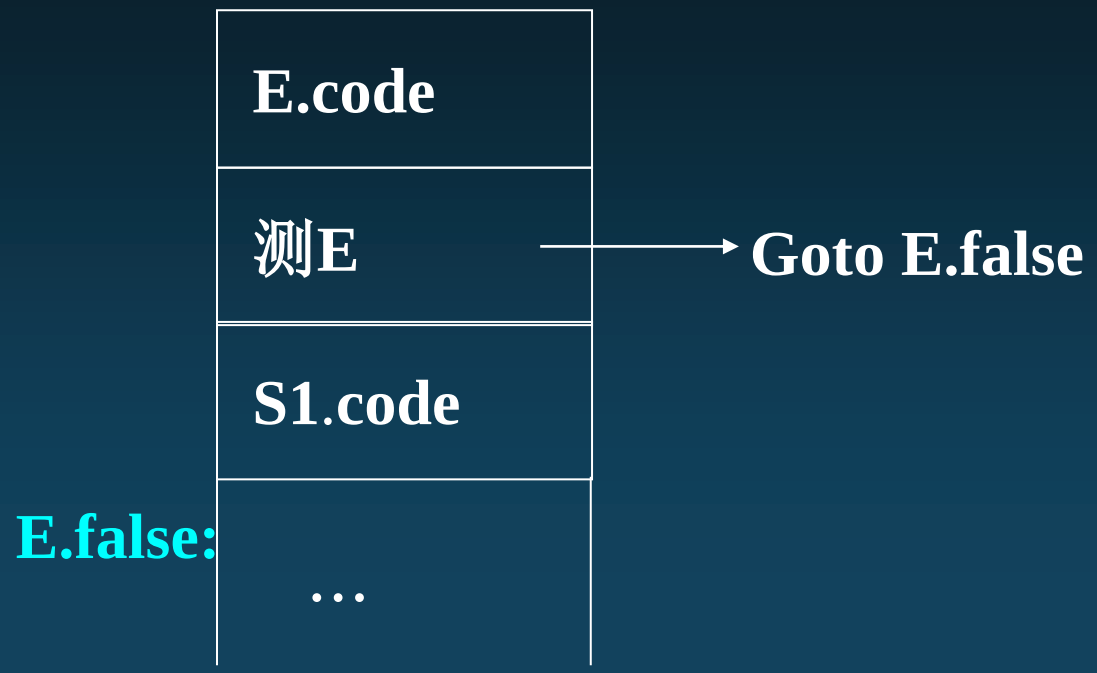
GEN(jF， _， _， E.false)|

GEN(E.true'：') |

S1.code|

GEN(E.false'：') }





E.false

S → if (E) then S1

- 条件语句①翻译的语义子程序

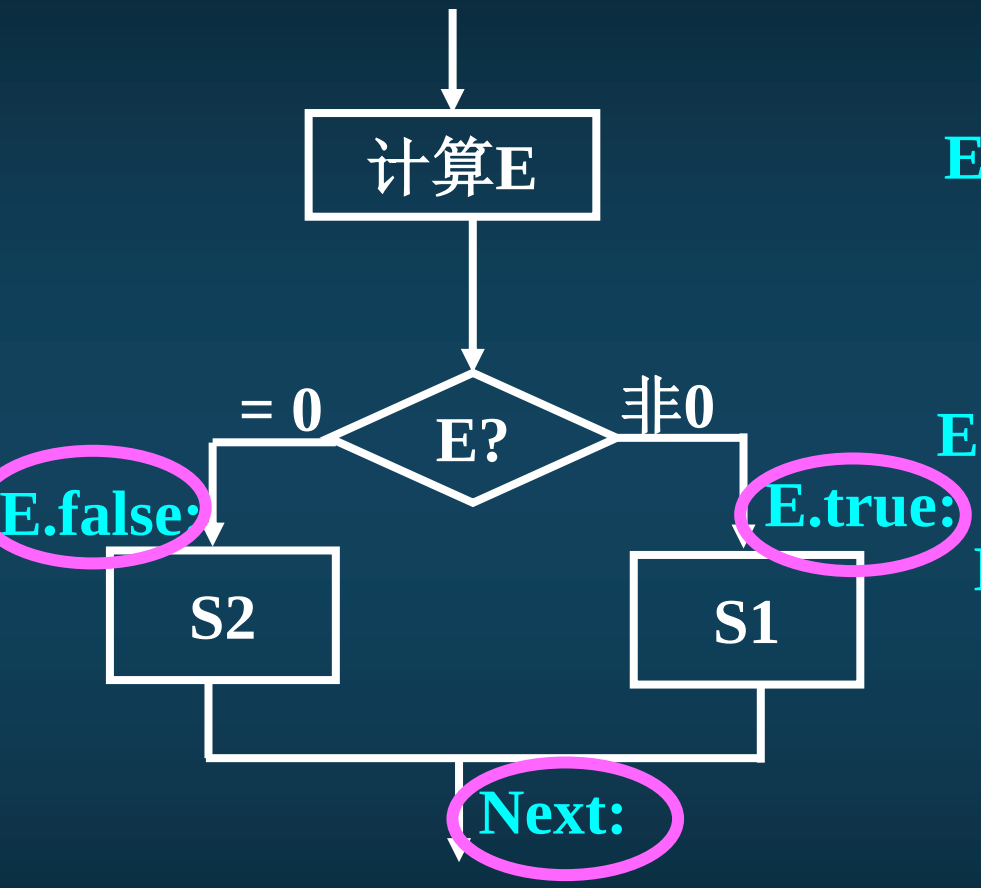
S →if E then S1

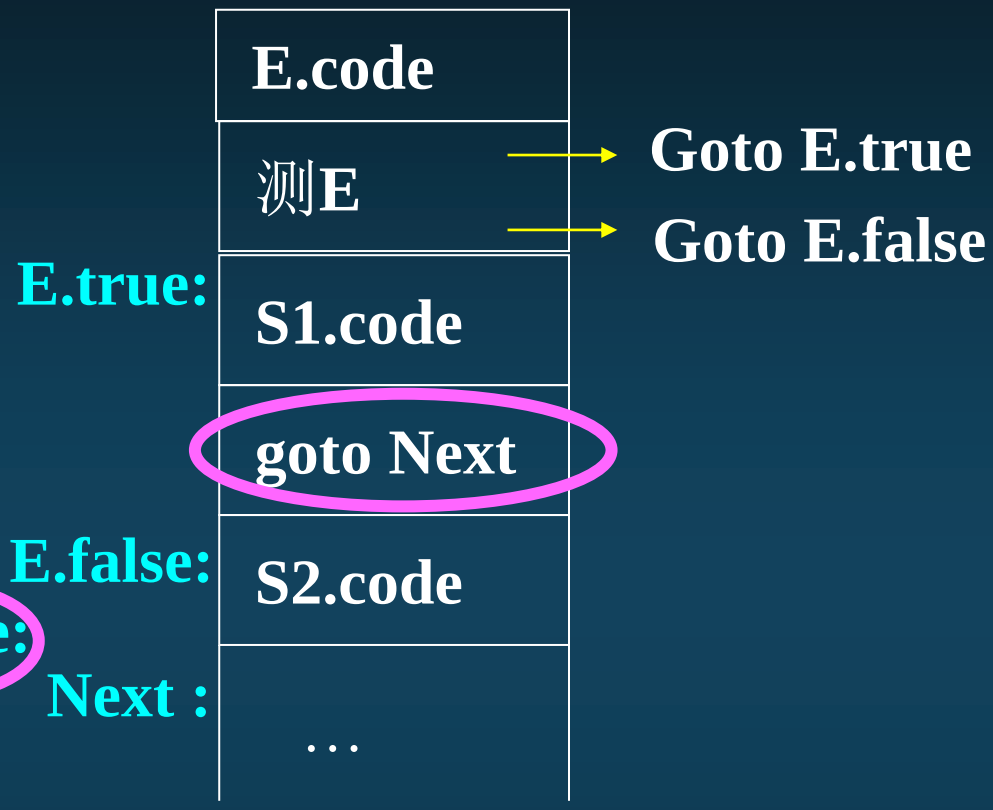
{ E.false=newlabel;

S.code=E.code|GEN(jF，_，_，E.false)|

S1.code|

GEN(E.false'：') }





E.true E.false Next

if (E) S1 else S2

- 条件语句②翻译的语义子程序

S →if E then S1 else S2

{ E.true=newlabel;

E.false=newlabel;

S.next=newlabel;

S.code=E.code | GEN(jT， _， _， E.true) |

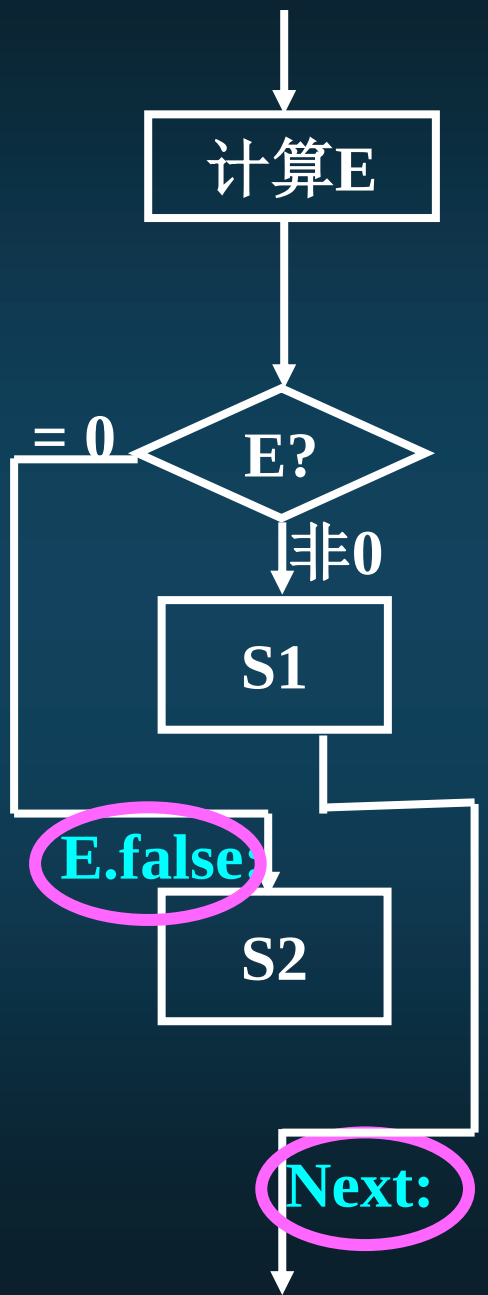
GEN (jF， _， _， E.false) |

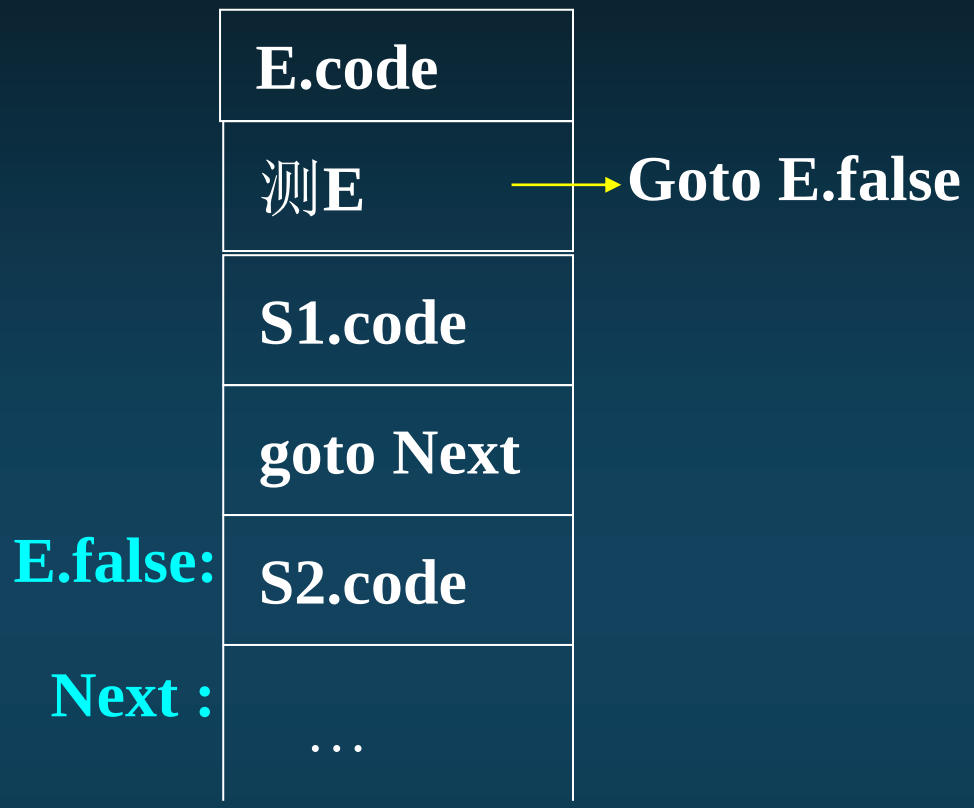
GEN (E.true'：'| S1.code |

GEN (j， _， _， S.next) |

GEN (E.false'：') |

S2.code|GEN(S.next '：') }





E.false Next

if (E) S1 else S2

- 条件语句②翻译的语义子程序

S →if E then S1 else S2

{ E.false=newlabel;

S.next=newlabel;

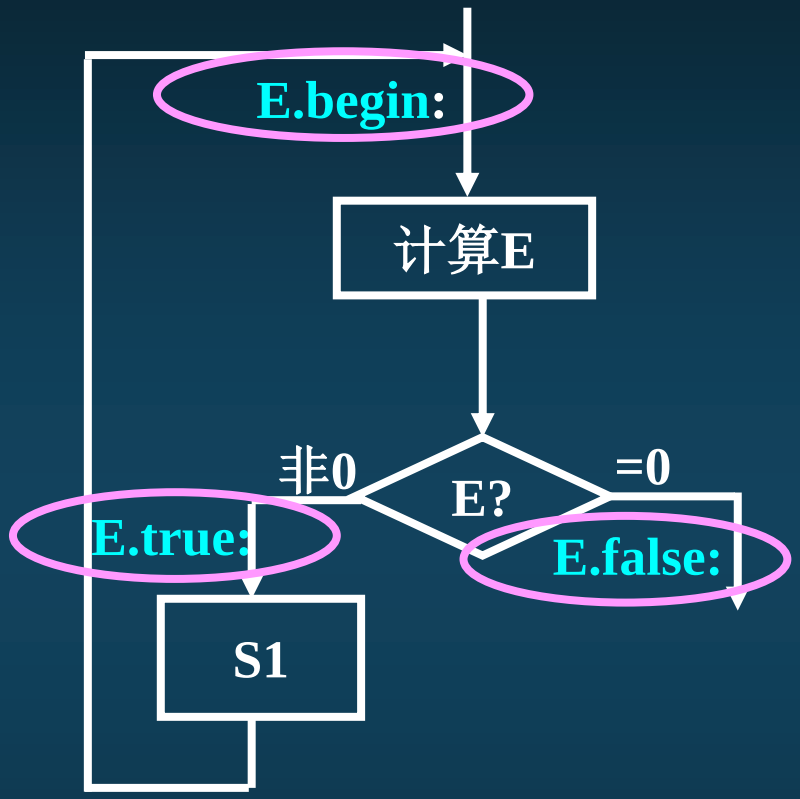
S.code=E.code | GEN (jF， _， _， E.false) |

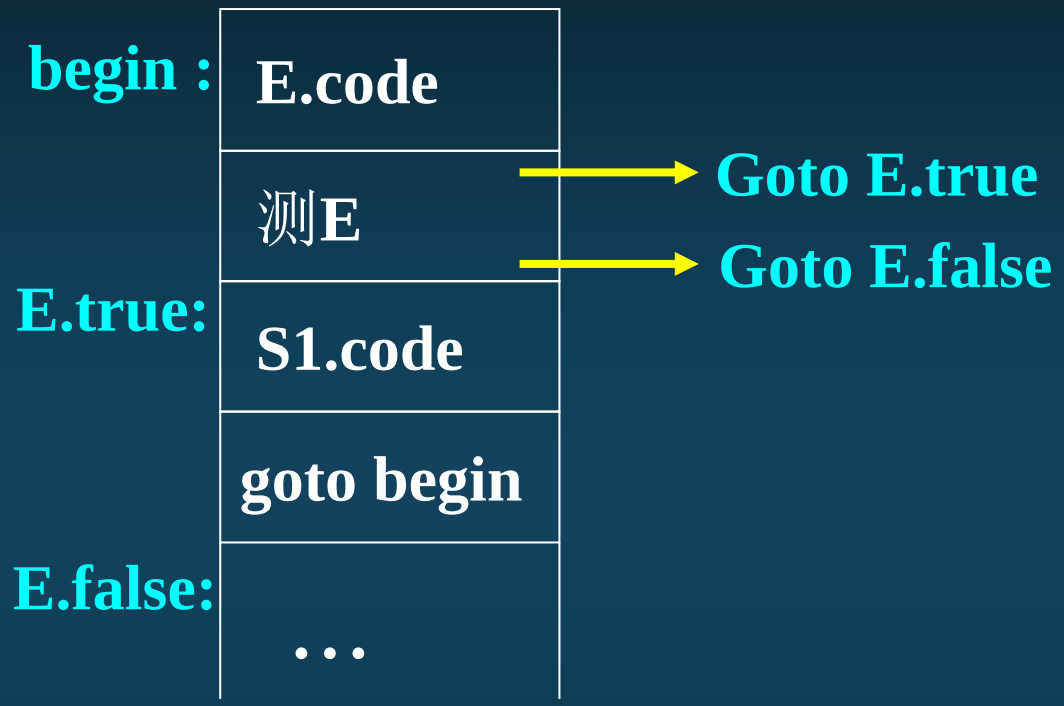
S1.code |

GEN (j， _， _， S.next) |

GEN (E.false'：') |

S2.code|GEN(S.next '：') }





begin E.true E.false

while (E) S1

- 条件语句③翻译的语义子程序

S → while (E) S1

{ E.true=newlabel;

E.false=newlabel;

S.begin=newlabel;

S.code=GEN(S.begin'：') | E.code |

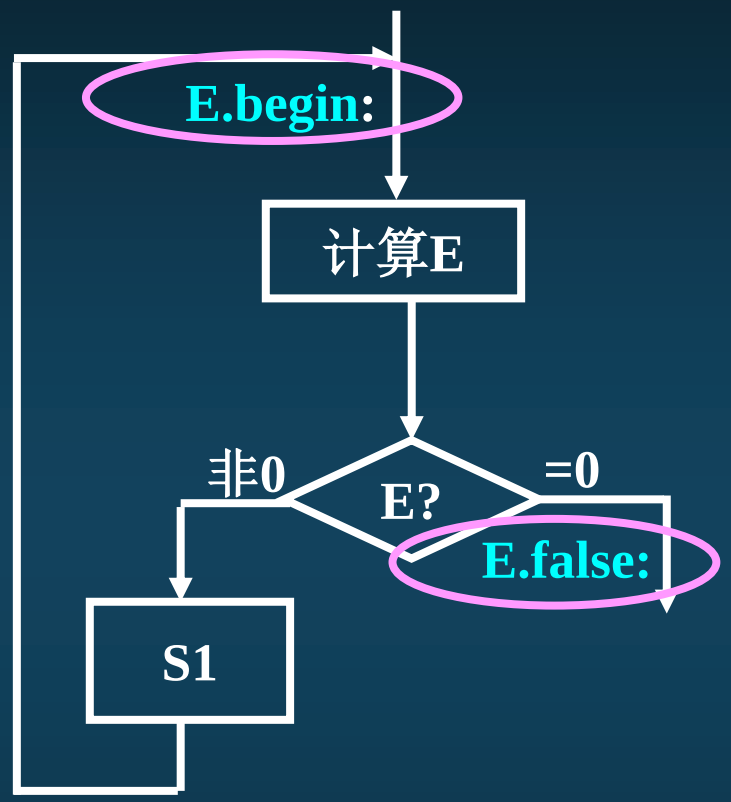
GEN(jT，_，_，E.true) |

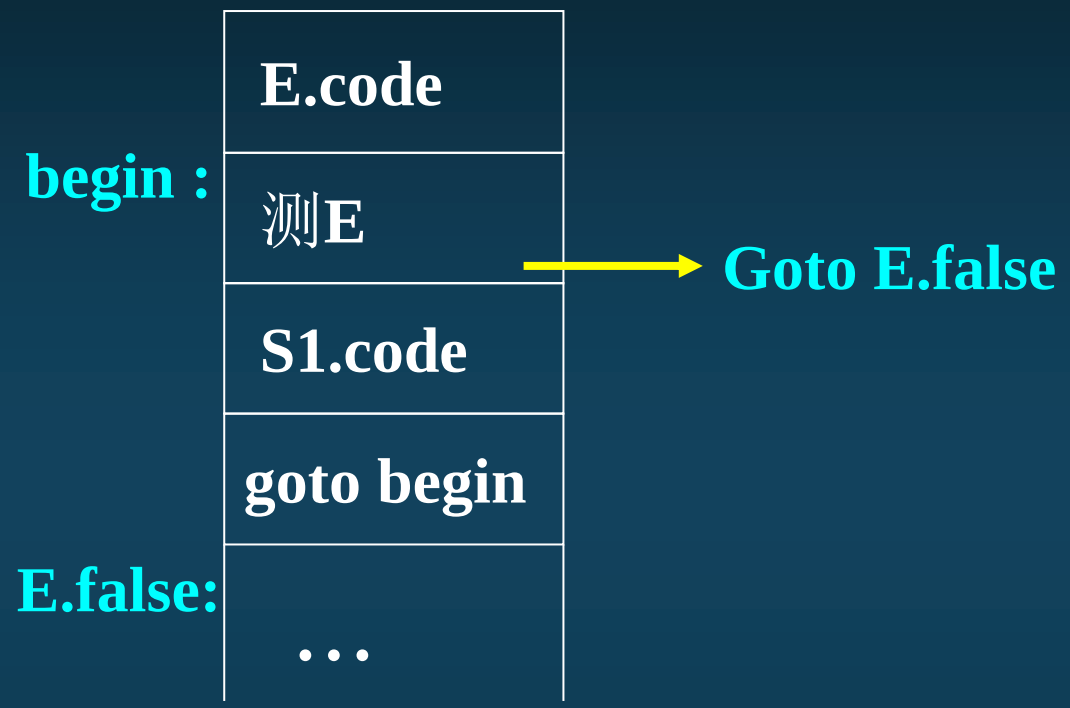
GEN(jF，_，_，E.false) |

GEN(E.true'：') | S1.code |

GEN(j，_，_，S.begin)|

GEN(E.false'：') }





begin E.false

while (E) S1

- 条件语句③翻译的语义子程序

S → while (E) S1

{ E.false=newlabel；

S.begin=newlabel；

S.code=GEN(S.begin'：') | E.code |

GEN(jF，_，_，E.false) |

S1.code |

GEN(j，_，_，S.begin)|

GEN(E.false'：') }

例6.2 将下面的C语言伪代码翻译为四元式形式的中间代码

```
if (A>B && B>C )
    S1
else
    S2
```

用到的语法：

$S \rightarrow$ if C S else S

$C \rightarrow C$ && $T|T$

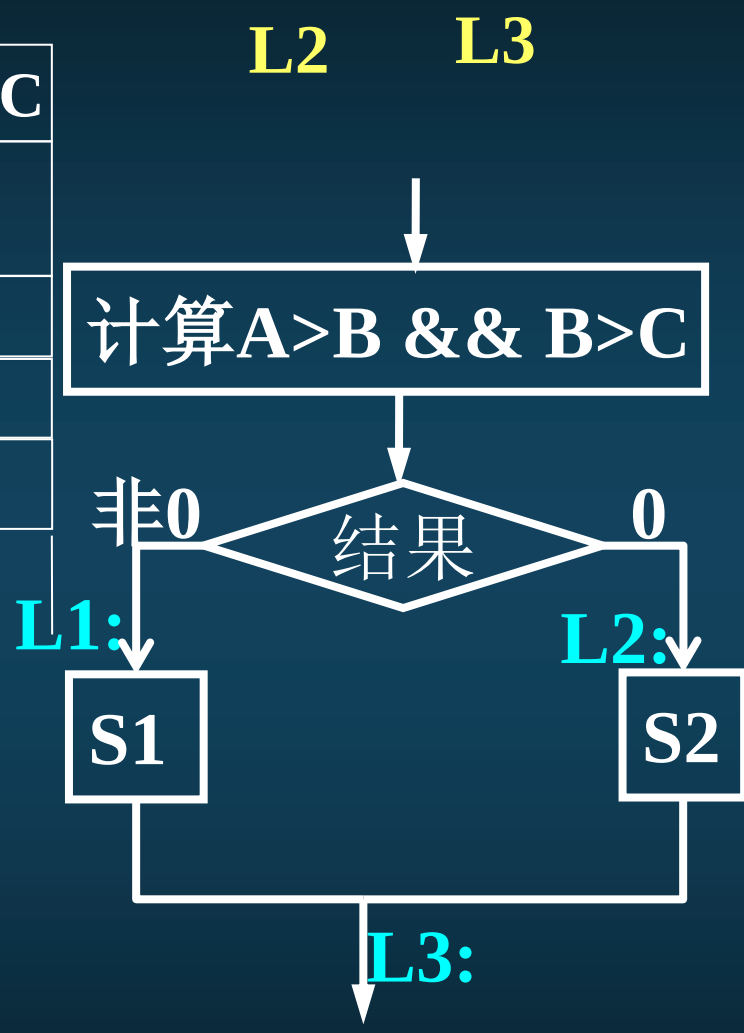
$T \rightarrow id>id$

例6.2 if (A>B && B>C) S1 else S2

10: (> A B T1)
14: (> B C T2)
18: (&& T1 T2 T3)
20: (J_T T3 _ 28)
24: (J_F T3 _ 36)
28: S1.code
32: (J _ _ 40)
36: S2.code
40: ……
44:

	A>B && B>C (L1)
	JT L1 JF L2
L1:	S1.code
	goto L3
L2:	S2.code
L3:	…

N	D	A
L1	1	28
L2	1	36
L3	1	40



6.6.3 控制流类语句翻译

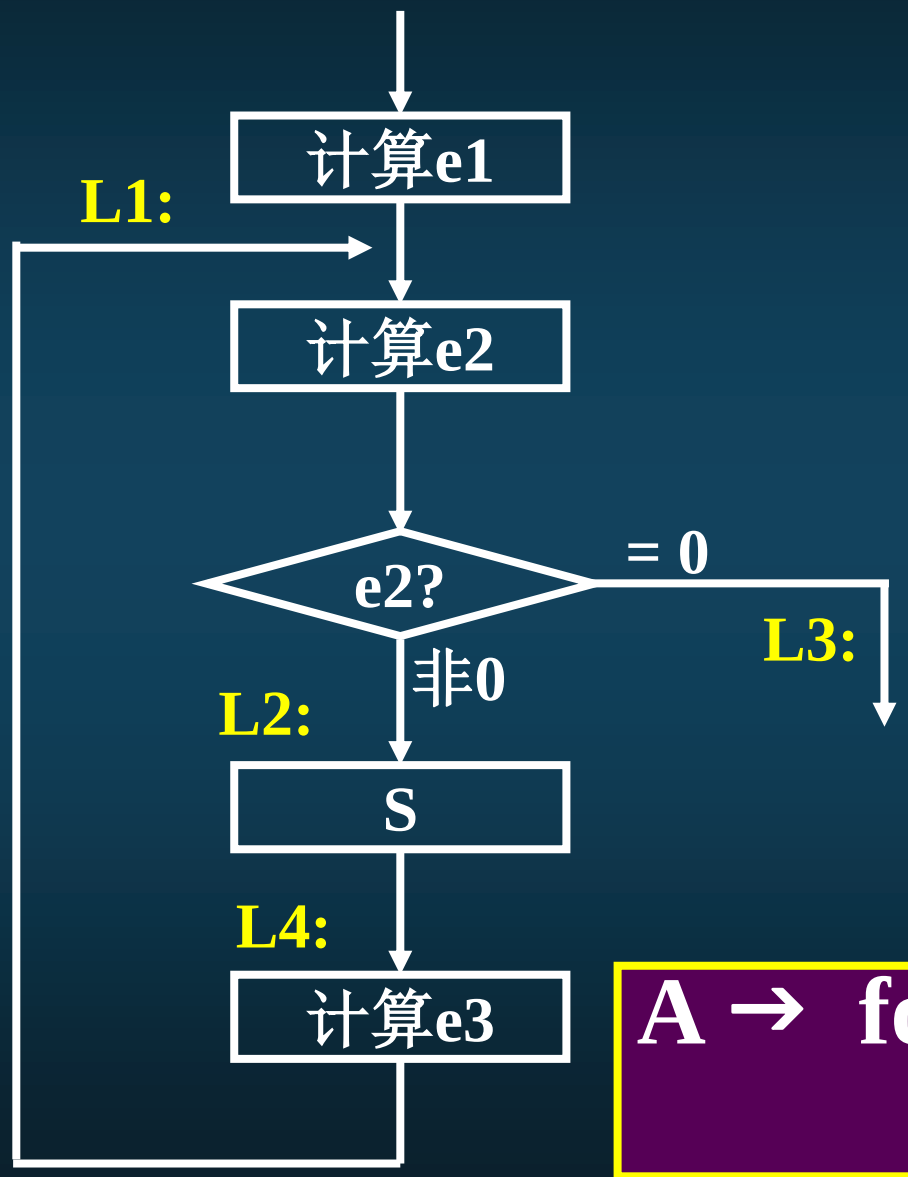
一. 语句标号与拉链返填技术

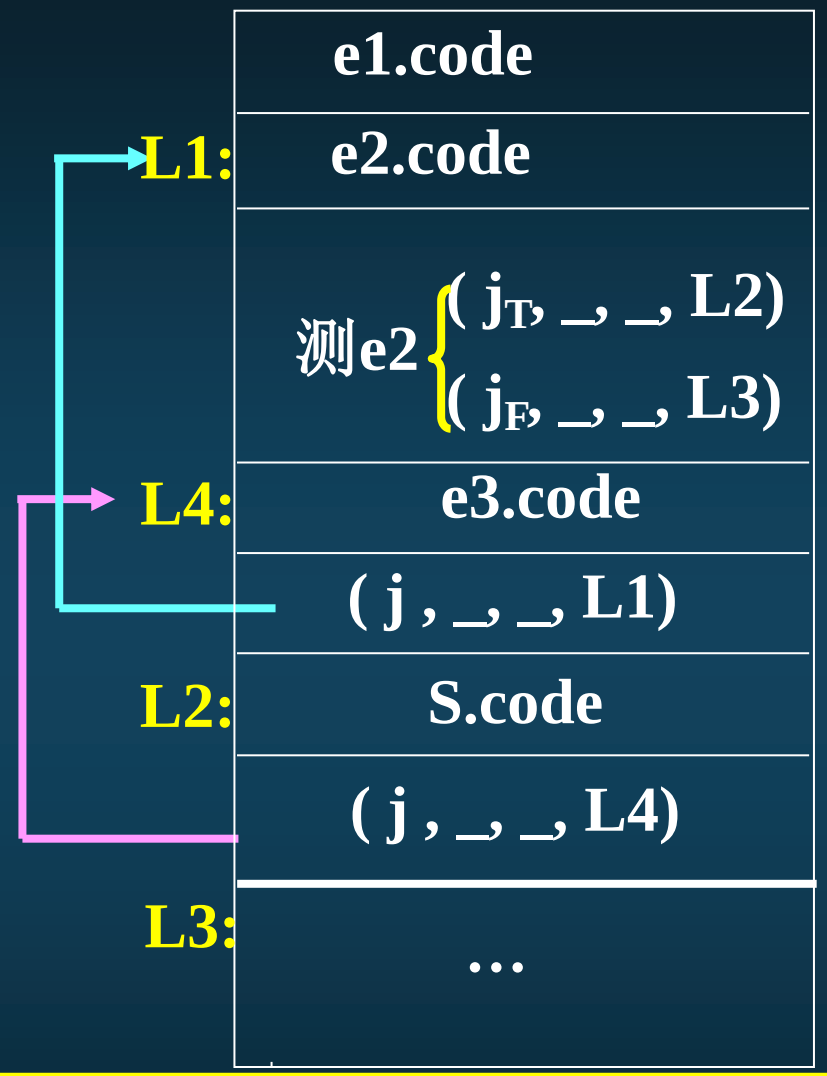
二. 条件语句的翻译

三. 循环语句的翻译

四. 多分支语句的翻译

三. 循环语句的翻译





A → for (e1 ; e2 ; e3) S

L1 L4 L2 L3

循环语句的翻译的语义子程序

A → for (e1 ; e2 ; e3) S

{ L1=newlabel; L2=newlabel;

L3=newlabel; L4=newlabel;

A.code=e1.code |GEN(L1': ') |

e2.code |

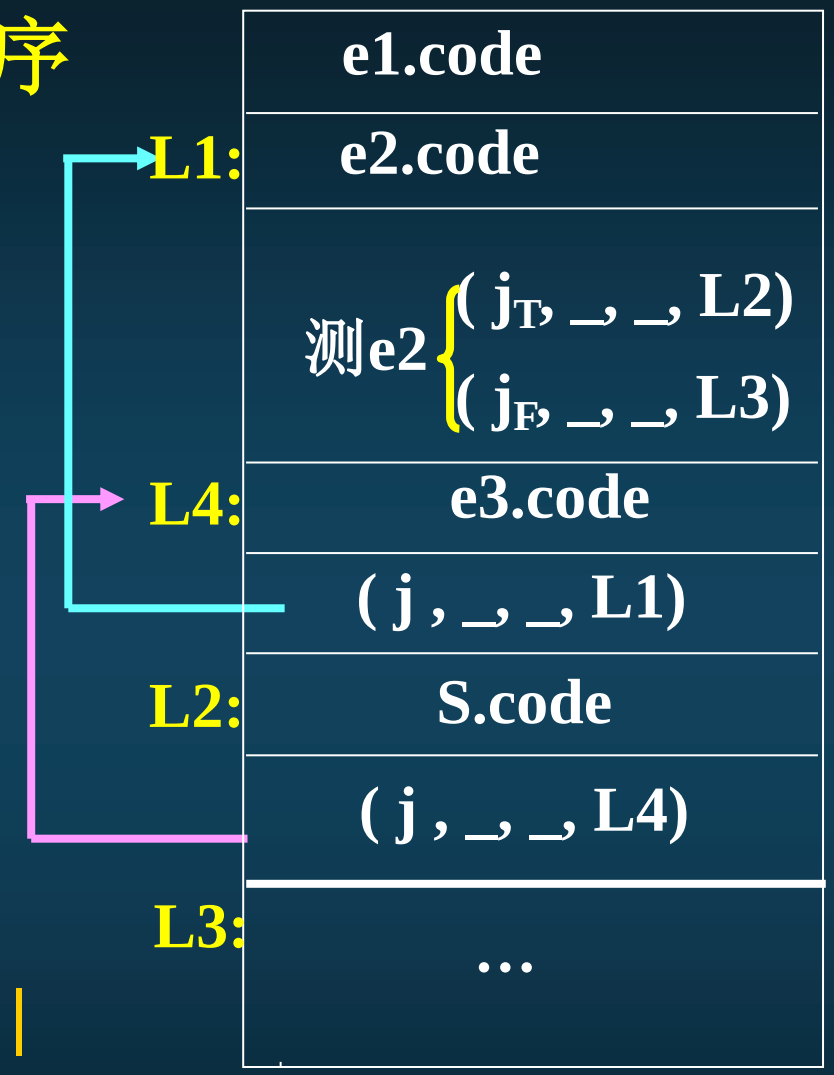
GEN(jT, _, _, L2) |

GEN(jF, _, _, L3) |

GEN(L4': ') | e3.code |

GEN(j, _, _, L1) |GEN(L2': ')

| S.code |GEN(j , _, _, L4) | GEN(L3': ')}



给出如下C程序段的目标代码结构：

…

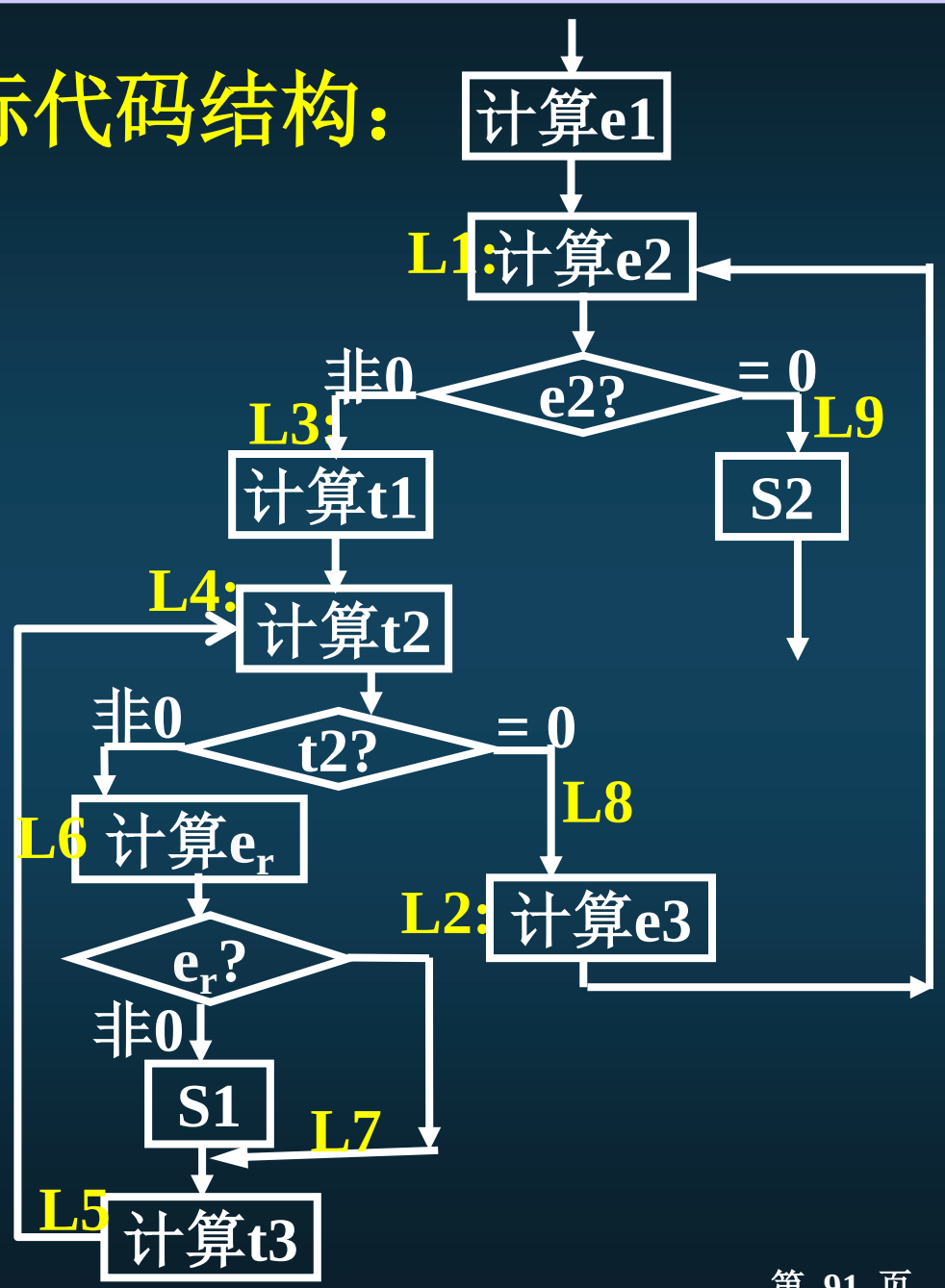
for (e1; e2; e3;)
L1 L2

for (t1; t2; t3;)
L3 L4 L5

if (e_r) S_1;
L6 L7

L8 L9 S_2;

…



	e1.code
L1:	e2.code
	测e2值 (j_T , , , L3) (j_F , , , L9)
L2:	e3.code
	(j , , , L1)
L3:	t1.code
L4:	t2.code
	测t2值
	(j_T , , , L6) (j_F , , , L8)

L5:	t3.code
	(j , , , L4)
L6:	e_r.code
	测e_r值 (j_F , , , L7) S1.code
L7:	(j , , , L5)
L8:	(j , , , L2)
L9:	S2.code

for (e1; e2; e3;)
L1 L2
for (t1; t2; t3;)
L3 L4 L5
if (e_r) S_1;
L6 L7
S_2;
L8 L9

6.6.3 控制流类语句翻译

一. 语句标号与拉链返填技术

二. 条件语句的翻译

三. 循环语句的翻译

四. 多分支语句的翻译

四. 多分支语句的翻译

$A \rightarrow$ switch (e)

{ case c_1 : S_1 break ;

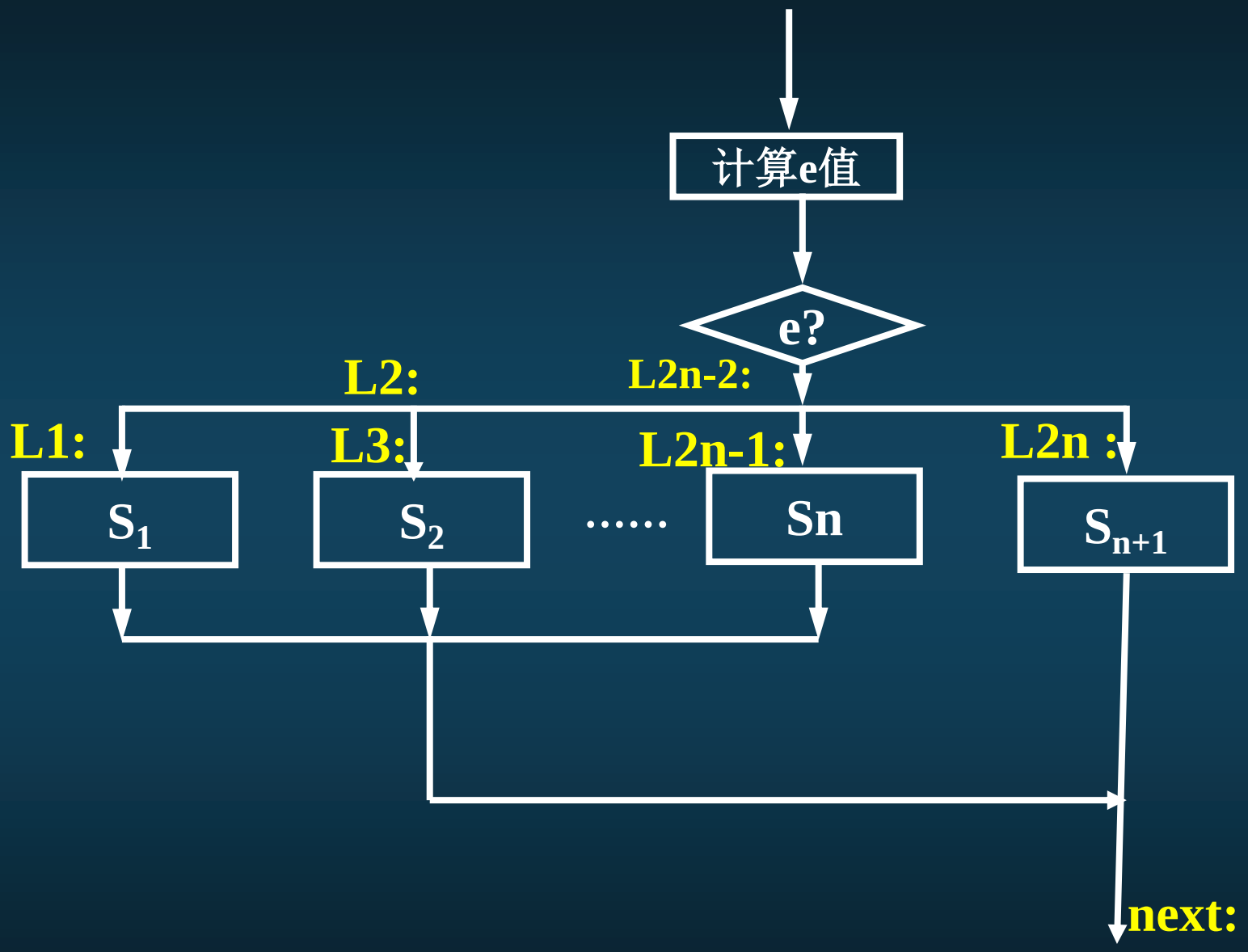
case c_2 : S_2 break ;

……

case c_n : S_n break ;

default : S_{n+1}

}


计算e值
e?
L2:
L2n-2:
L1:
L3:
L2n-1:
L2n :
S1
S2
……
Sn
Sn+1
next:

四. 多分支语句的翻译

$A \rightarrow$ switch (e)

{ case c_1 : S_1 break ;

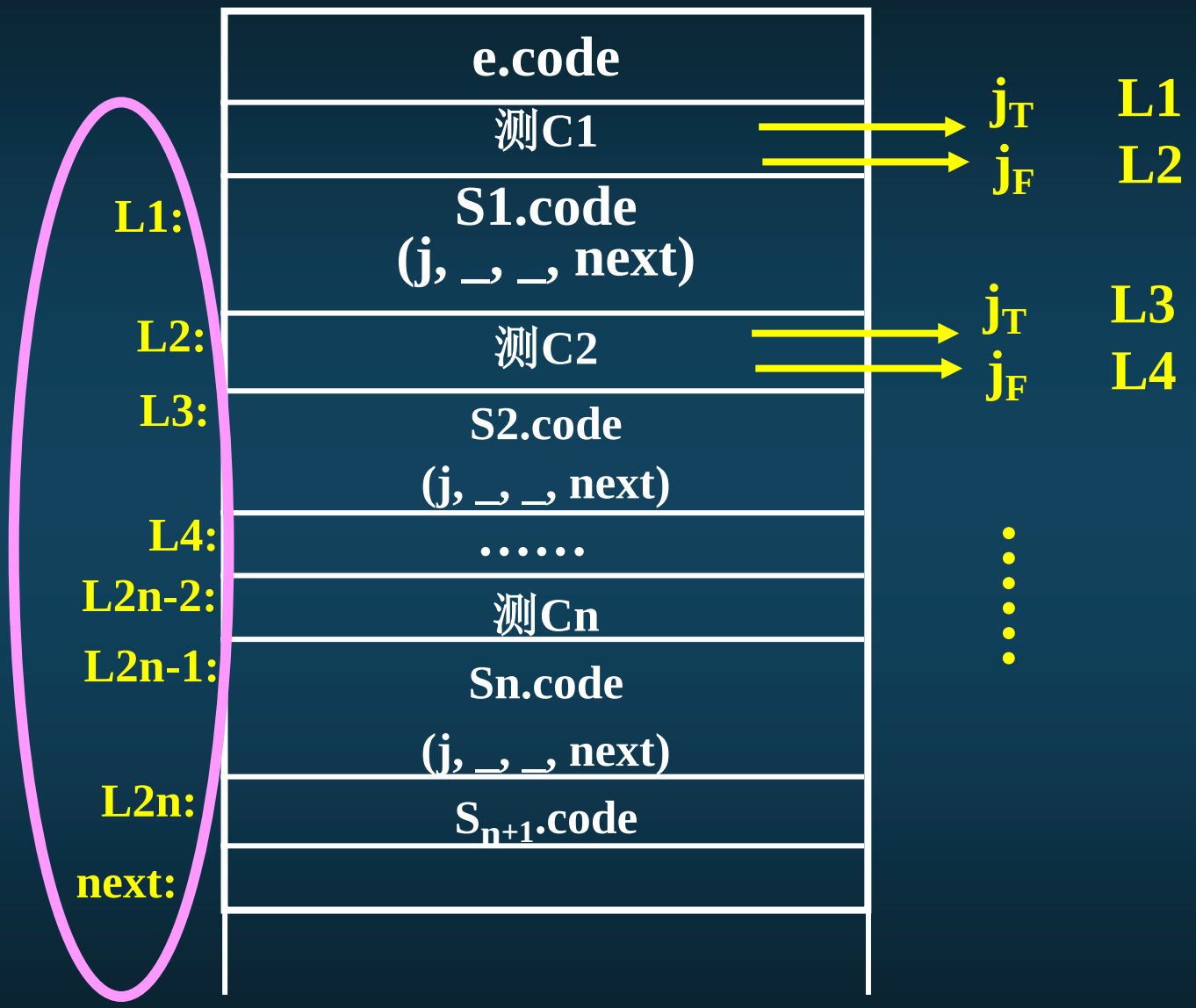
case c_2 : S_2 break ;

……

case c_n : S_n break ;

default : S_{n+1}

}


e.code
测C1
S1.code
(j, _, _, next)
测C2
S2.code
(j, _, _, next)
……
测Cn
Sn.code
(j, _, _, next)
S_{n+1}.code
L1:
L2:
L3:
L4:
L2n-2:
L2n-1:
L2n:
next:
j_T L1
j_F L2
j_T L3
j_F L4

$A \rightarrow$ switch (e)

{ case c_1 : S_1 break ;

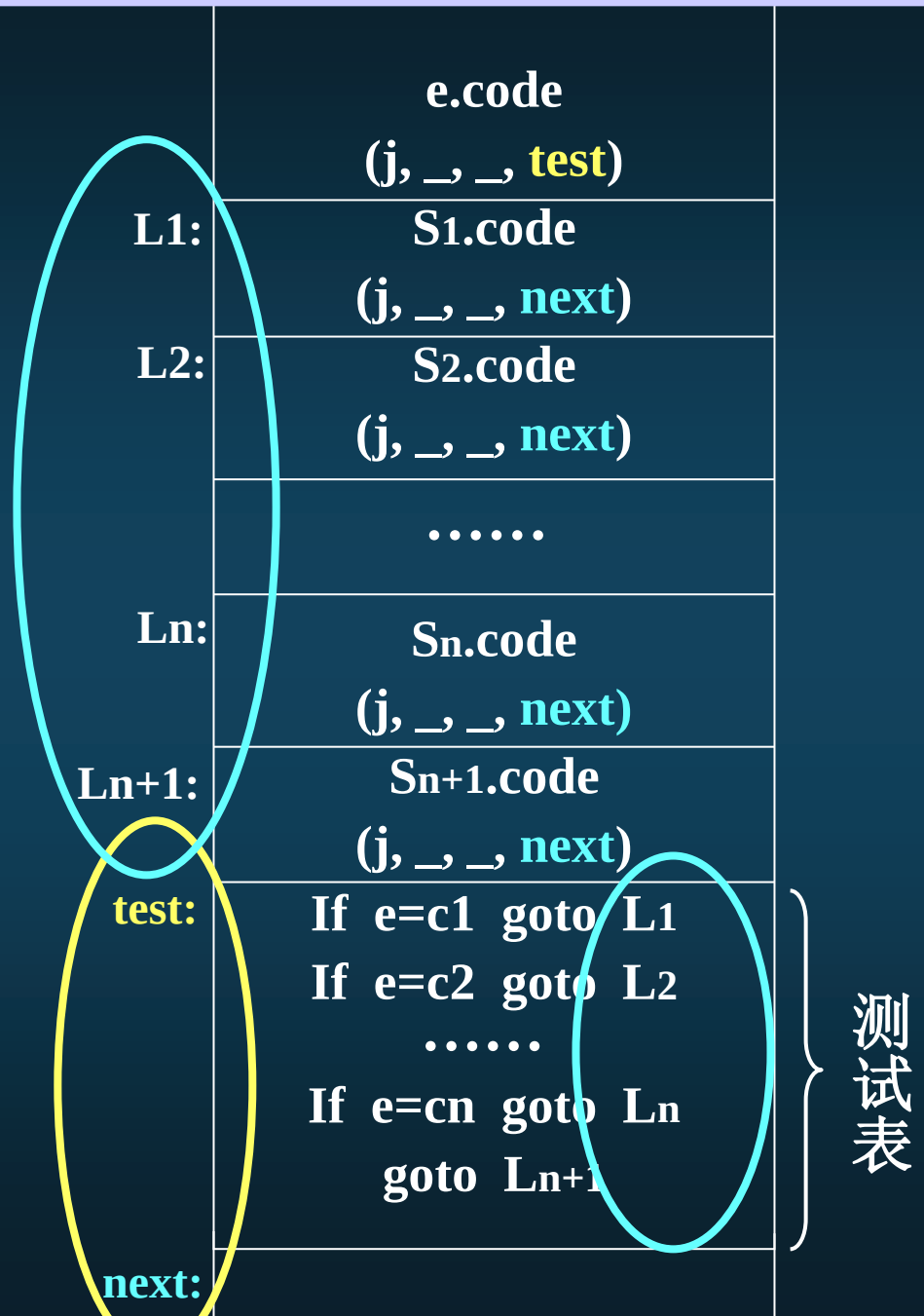
case c_2 : S_2 break ;

……

case c_n : S_n break ;

default : S_{n+1}

}


e.code
(j, _, _, test)
L1:
S1.code
(j, _, _, next)
L2:
S2.code
(j, _, _, next)
……
Ln:
Sn.code
(j, _, _, next)
Ln+1:
Sn+1.code
(j, _, _, next)
test:
If e=c1 goto L1
If e=c2 goto L2
……
If e=cn goto Ln
goto Ln+1
测试表
next:

第 6 章 语义分析与中间代码生成

6.6.1 说明类语句的翻译

6.6.2 赋值语句与表达式翻译

6.6.3 控制流类语句翻译

6.6.4 数组说明与数组元素引用的翻译

6.6.5 过程、函数说明和调用的翻译

一. 数组说明

$$\text{type}\quad A\ [d_1]\,[d_2]\ldots[d_n]\ ;$$

数组类型　数组名　数组维数、每维大小、数组体积

n: d_i个数

d_i的值

$d_1 * d_2 * \ldots * d_n$

$$\text{type}\quad A\ [d_1 .. d_1']\,[d_2 .. d_2']\ldots[d_n .. d_n']\ ;$$

数组说明的语义处理

填表，登记数组的属性信息。

type
name
dim
dim_value
vol

符号表

公共、等长信息
（符号表）

与计算数组元素地址有关、不等长信息
（信息向量表）

例6.3设有说明

int a[2][2];

float b[4];

namelist

name	kind	type	addr
a	array	I	
b	array	R	

信息向量表

2	a
2	
2	
2*2	
C_a	
a_0	
4	b
1	
4*1	
C_b	
b_0	

信息向量表

第一维大小
第二维大小
…
第n维大小
维数
体积 $=d_1*d_2*\ldots*d_n$
计算数元地址不变部分C
数组第一个元素地址a
…

return

若为上下界需计算：

a(2..10)→10-2+1=9

何时计算？

动态数组如何处理？

二. 数组元素引用

type A $[d_1]$ $[d_2]$… $[d_n]$ ； 数组说明

A $[i_1]$ $[i_2]$… $[i_n]$ 数组引用

** 不能整体引用，仅对单一数组元素引用。

➢ 数组元素引用的语义处理

语义检查： 类型匹配；下标越界检查；

产生代码：数组元素地址计算的中间代码。

数组元素地址计算编译时能否完成？

一般不能

∵ A $[i_1]$ $[i_2]$… $[i_n]$ 中 i_k多为表达式

如， a[i+j-1][j++] 运行时得到下标值

涉及数组元素地址计算的有关知识

- 数组存储方式 (线性连续)
  - 按行存储
  - 按列存储

例如， int a[2][2];

按行

a_0	a[0][0]
	a[0][1]
	a[1][0]
	a[1][1]

按列

a_0	a[0][0]
	a[1][0]
	a[0][1]
	a[1][1]

$a[0][1]_{add} = a_0 + 1*byte$

$a[0][1]_{add} = a_0 + 2*byte$

- 数组元素地址计算

据存储方式，通过下述规则计算数组元素位置

$$\boxed{(\text{顺序号}) + a_0}$$

- 对一维数组 int a[n]; // 默认下标下界=0

则 $a[i]_{addr} = a_0 + (i)$

(若数组下标下限=1，则$a[i] = a_0 + (i-1)$)

- 对二维数组 int a[n][m] ;

则 $a[i][j]_{addr} = a_0 + (i * m + j)$

若数组下标下限=1，则

$$a[i][j]_{add} = a_0 + ((i-1) * m + (j-1))$$

- 对n维数组 int a[d_1] [d_2]… [d_n];

则 a[i_1] [i_2] … [i_n] $_{add}$

$$= a_0 + (i_1 * d_2 * d_3 * \ldots * d_n + i_2 * d_3 * d_4 * \ldots * d_n + \ldots + i_{n-1} * d_n + i_n)$$

含i_k,是可变部分，程序运行时方可知。

进一步考虑更一般的情况：若数组下标下限≠0

若数组下标下限=1

则 $a[i_1][i_2]\ldots[i_n]_{add}$

$$= a_0 + ((i_1-1)*d_2*d_3*\ldots*d_n + (i_2-1)*d_3*d_4*\ldots*d_n + \ldots + (i_{n-1}-1)*d_n + (i_n-1))$$

变换

V

$$= a_0 + i_1d_2d_3\ldots d_n + i_2d_3d_4\ldots d_n + \ldots + i_{n-1}d_n + i_n - (d_2d_3\ldots d_n + d_3d_4\ldots d_n + \ldots + d_n + 1)$$

不变

C

返回

$a[i_1][i_2]\ldots[i_n]_{add}$

V

$$= a_0 + i_1 d_2 d_3 \ldots d_n + i_2 d_3 d_4 \ldots d_n + \ldots + i_{n-1} d_n + i_n$$

$$- (d_2 d_3 \ldots d_n + d_3 d_4 \ldots d_n + \ldots + d_n + 1)$$

不变

C

$$= \underbrace{a_0 - C}_{\text{conspart}} + \underbrace{V}_{\text{varpart}}$$

$$= CUN + V$$

编译时计算,填入内情向量表

据数元引用情况，产生计算的中间代码，程序运行时算出。

信息向量表

数组元素引用目标结构

V.code $V_{值} \Rightarrow V$	计算V的中间代码
$CUN = a_0 - C$	计算 CUN
$T = CUN + V$	数组元素地址 $\Rightarrow$ T

例6.4 设有说明

…

int a[10][20]

…

s= a[x][y];

…

namelist

name	kind	type	addr
a	array	I	
…			

V.code
$V_{值} \Rightarrow V$
$CUN = a_0 - C$
$T = CUN + V$

$a[x][y]_{addr}$•code

① t1 = x * 20

② t1 = t1 + y

③ t2 = 4 * t1

④ t3 = a_0 – 84

⑤ t4 = t3[t2]

⑥ s = t4

内情向量表

10
20
2
10*20=200
C=(20+1)*4=84
a_0
…

C的计算

- 动态数组？

type A $[d_1]$ $[d_2]$… $[d_n]$； （d_i是变量）

（每维大小、体积等信息程序运行时确定）

- 动态数组处理

内情向量表在程序运行时建立，编译时仅分配内情向量表所占存储空间(据维数)。在运行时， d_i确定后，计算体积、计算C、为数组分配存储空间且将这些信息登录内情向量表。

∴ 动态数组：产生此部分代码 + 静态数组元素引用

动态数组处理目标结构

取$d_i \Rightarrow$ 内情向量表code
计算体积、C的code
为数组分配存储的code
V.code $V_{值} \Rightarrow V$
$CUN = a_0 - C$
$T = CUN + V$

动态数组说明

V.code → 计算V的中间代码

$CUN = a_0 - C$ → 计算不变部分 $\Rightarrow$ CUN

$T = CUN + V$ → 数组元素地址 $\Rightarrow$ T

第 6 章 语义分析与中间代码生成

6.6.1 说明类语句的翻译

6.6.2 赋值语句与表达式翻译

6.6.3 控制流类语句翻译

6.6.4 数组说明与数组元素引用的翻译

6.6.5 过程、函数说明和调用的翻译

函数翻译 { 处理说明（定义）
处理调用

一. 函数说明的翻译

1） 函数及局部量信息登录符号表；

2） 记录函数中局部量的作用域(Level)；

例6.5 设有C函数

```
fun1( int a,b)
 {
   int   c;
   c=a+b ;
   return ( c )
 }
```

namelist

name	kind	type	len	offset	addr
fun1	函数	I			→ code
a	形参	I	4	0	
b	形参	I	4	4	
c	V	I	4	8	

二. 函数调用的翻译

G: S → call id (Elist)

Elist → Elist，E | E

其中： id: 函数名；

E: 表达式。

Elist: 实参表；

- 函数调用的语义处理：

① 检查所调用的过程或函数是否定义；与所定义的过程或函数的类型、实参与形参的数量、顺序及类型是否一致；

② 给被调过程或函数分配活动记录所需的存储空间；

③ 传送实参并计算；

④ 加载调用结果和返回地址，恢复主调用过程或函数的继续执行；

⑤ 转向相应的过程或函数。

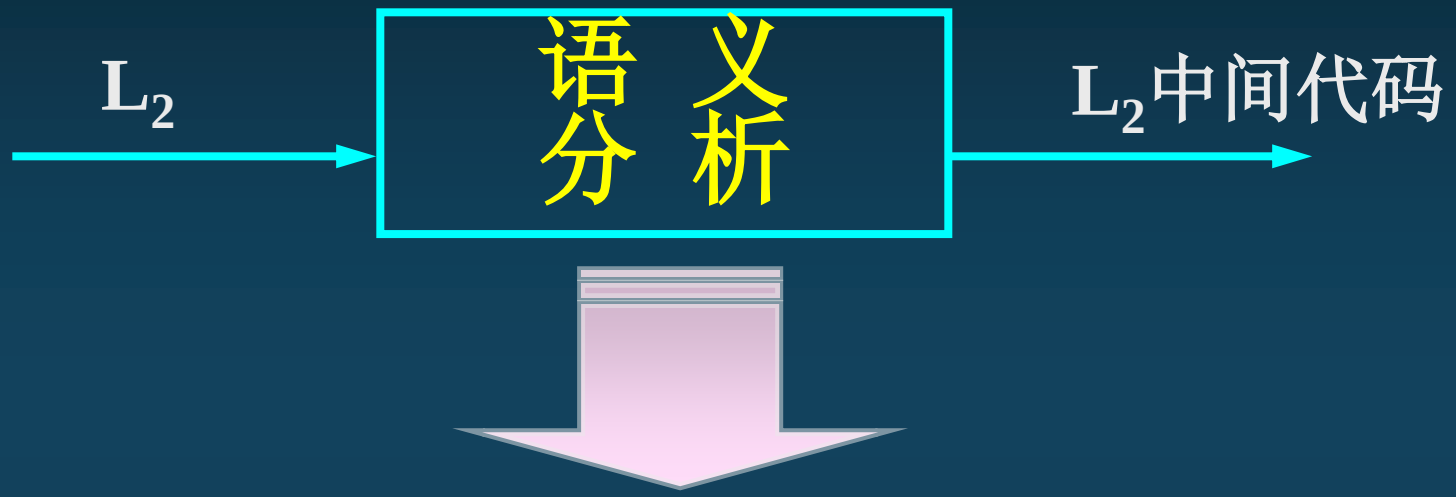
- 函数调用返回的语义处理

① 返回一个或多个结果值存于指定位置；

② 恢复调用过程的活动记录；

③ 生成一条转移指令（返回）；

语义分析与中间代码生成总结



根据语义进行语义检查和代码生成

对L_2扫描的顺序性；产生的代码执行的线性性；

语义映射的准确性；

语义分析与中间代码生成

说明性语句 → 登录编译信息、为可执行语句提供语义检查和语句翻译的信息依据。

可执行语句 —设计→ 目标代码结构 —映射→ 中间代码

动态语义 → 设计

语义动作 → 映射

第六章 结束

(1)给定文法G：E→E+T|T

T→T*F|F

F→i|(E)

则L(G)中的一个句子i+i+(i*i)*i的逆波兰表示为_____。

A) iii*i++　　B) ii+iii**+

C) ii+ii*i*+　　D) A、B、C都不正确

(2) 中间代码生成时所依据的是_____。

A）语法规则　　B）词法规则

C）语义规则　　D）等价变换规则

(3)在编译程序中与生成中间代码的目的无关的是___。

A)便于代码优化　　B)便于存储空间的组织

C)便于移植　　D)便于生成目标代码

(4) 在语法制导翻译中不采用拉链-返填技术的语句是_____。

A）转向语句　　B）赋值语句

C）条件语句　　D）循环语句

判断正误

(1)使用语法制导翻译方法的编译程序能同时进行语法分析和语义分析。

(2)返填就是稍后填写转移指令的地址。

(3)对任何一个编译程序,产生中间代码是不可缺少的。

(4)三元式同间接三元式是等价的。

(5)语法制导翻译方法可用来产生各种中间代码，又可用来产生目标代码。

设有下列C语言的语句（注： Li是设定的语句标号,在注释中标识出所在位置）

```
 i=j=0;
for ( ; j>=0, i<=100; i++,j--)
                    //for (;L1: j>=0, i<=100; L2：  i++,j--)
      if (E1)  S1;          // if  L3: (E1) L4: S1;
 S2;                        //  L5: S2;
```

其中：S1和S2代表语句，对应的四元式分别为S1.code和S2.code；E1代表条件表达式，对应的四元式为E1.code。

1．给出该语句的四元式形式的目标代码结构，填入下表。

说明：无条件转移操作符用“j”表示；条件成立转移的操作符用“j_T”表示；条件不成立转移的操作符用“j_F”表示。

Adress	code	备注(可有可无)

2. 给出中间代码生成后对应该段代码填写的下列语句标号表内容（按填写标号表的实际顺序）。注：设编译器是一遍扫描的编译器。

语句标号 (Li)	定义与否标志 (1表示已定义，0反之)	Addr